HANDBUCH

Bedienungsanleitung Python-Interpreter

Aufruf, Schalter und Geräte von c8205_interp.py

Diese Anleitung zeigt, wie man Testprogramme für den Cellatron C 8205 schreibt, mit dem Python-Interpreter ausführt und die Ergebnisse liest: Aufruf und Schalter, Aufbau der Testdatei, Ein- und Ausgabegeräte, Endzeile, Protokoll und Fehlermeldungen. Sie richtet sich an Personen, die den Aufbau des Interpreters nicht kennen. Den Rechner selbst beschreiben die PDFs „Rechnerbeschreibung“ und „Bedienpult und Peripherie“.

Quellen	C8205-Interpreter-Python_Interpreter.md	0.03-006
	C8205-Interpreter-Durchführung_eines_Tests.md	0.03-009
Stand	13.10.2026	
Umfang	37 Seiten, A4 hoch	
Erzeugt mit	erzeuge_bedienungsanleitung_pdf.py	

> Maßgeblich bleiben die Quelldateien. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Versionen oben mit den Versionszeilen der Quelldateien übereinstimmen.

Lesehinweise

Gilt für: `c8205_interp.py`, Version **0.03-009** (Abfrage: `python3 c8205_interp.py --version`), zusammen mit den Wrappern `c8205_interpreter.sh` und `c8205_interpreter.bat` und dem Hilfsskript `erzeuge_leere_trommel.py`. Alle Beispiele dieser Anleitung sind beim Erzeugen des PDFs mit dieser Version ausgeführt worden, jeweils auf frisch erzeugter leerer Trommel; die Ausgaben sind übernommen, nicht von Hand geschrieben. Ändert sich der Interpreter (neuer Schalter, neues Token, neue Meldung), muss das PDF neu erzeugt werden.

Zielgruppe: Personen, die Testprogramme für den C 8205 schreiben und durchrechnen, ohne den Aufbau des Interpreters zu kennen. Kenntnisse über Oktalzahlen und Register werden nicht vorausgesetzt; Begriffe stehen im Glossar (Anhang A).

Abgrenzung: Die Anleitung erklärt die Bedienung des Interpreters, nicht den Rechner. Zum Rechner siehe die PDFs „Rechnerbeschreibung“ und „Bedienpult und Peripherie“; das Testformat im Wortlaut der Quelle steht im PDF „Durchführung eines Tests“, eine Einführung in das Schreiben von Tests im „Testhandbuch“.

Legende der Darstellung

Darstellung	Bedeutung
`$ befehl`	Eingabe im Terminal (Befehl, den Sie tippen), in Fettschrift.
Normale Zeile	Ausgabe des Interpreters.
Dunkelgelb	Ausgabe des Rechners über das Schreibwerk (Kanal K2), ebenfalls fett. Im Schwarz-Weiß-Druck erkennen Sie sie an der Zeile selbst (z. B. „Jahr 1972“) und am Hinweis im Text.
Rot	Fehlermeldung zu einem Gerät. Jede solche Meldung beginnt mit „FEHLER:“, damit sie auch ohne Farbe auffällt. Meldungen der Vorab-Prüfungen (Aufruf, Testdatei, Trommeldatei) erscheinen ohne Farbe, z. B. als „FORMALE PRUEFUNG NEGATIV“ oder „FEHLER: Testdatei nicht gefunden“.
Hinweis / Achtung	Blauer Balken: nützliche Ergänzung. Orangefarbener Balken: Stelle, an der Tests häufig scheitern oder an der Hardware und Interpreter abweichen.

Schreibweisen

- Dateinamen, Schalter und Eingaben stehen in `Festbreitenschrift`.
- Zahlen in Testdateien und Registerwerten sind **oktal** (Ziffern 0–7). Wo Ziffern dezimal gemeint sind, steht es dabei.
- Kanäle heißen immer K0 bis K3. Geräte heißen Lochstreifenleser (Kürzel Leser), Lochstreifenstanzer (Stanzer) und Schreibwerk (SW).
- **Testdatei** ist die Datei, die Sie dem Interpreter geben. **Trommelabschnitt** ist ihr erster Teil (`Trommeldatei:`: die Zellen, die der Test vor dem Start auf die Trommel schreibt), **Tastenfolge** ihr zweiter Teil (die Bedienhandlungen). **Trommeldatei** ohne Zusatz ist die persistente Datei `trommel.txt`, der Speicher des Interpreters.
- Die Zahl hinter „Knoten:“ in der Endzeile ist fassungsabhängig und kein Sollwert. Ordnernamen und Pfade der Beispiele sind frei gewählt.

Verwandte PDFs: Rechnerbeschreibung · Bedienpult und Peripherie · Testhandbuch · Durchführung eines Tests · Einzelbefehlsliste · Kodetabellen · Knotengraph · Lochstreifen-Hilfsprogramme · Einstieg für Neueinsteiger · Projektüberblick (Verweise: Anhang C).

Inhaltsverzeichnis

1 Zu dieser Anleitung . . . 4
1.1 Was der Interpreter tut . . . 4
1.2 Wie die Anleitung aufgebaut ist . . . 4
1.3 Stand und Neuerungen . . . 4

2 Erste Schritte . . . 4
2.1 Voraussetzungen . . . 4
2.2 Ordner . . . 5
2.3 Die Trommeldatei erzeugen . . . 5
2.4 Der erste Lauf . . . 6

3 Tests schreiben . . . 6
3.1 Trommelabschnitt . . . 7
3.2 Testmarken . . . 7
3.3 Tastenfolge . . . 10
3.4 Prüfungen (optional) . . . 12
3.5 Formale Prüfung vor dem Start . . . 12
3.6 Geräte in der Testdatei . . . 13
3.7 Die Trommel zwischen den Tests . . . 13

4 Interpreter starten . . . 14
4.1 Kommandozeile . . . 14
4.2 Mehrere Tests, Protokollort . . . 16
4.3 Wrapper . . . 16
4.4 Rückkehrcodes . . . 16

5 Ein- und Ausgabe . . . 17
5.1 Schreibwerk-Ausgabe (K2) . . . 17
5.2 Lochstreifenleser (K1, K2) . . . 18
5.3 Lochstreifenstanzer (K1) . . . 18
5.4 Pult-Tastatur und Schreibwerk-Eingabe (K0) . . . 18

6 Ergebnisse lesen . . . 19
6.1 Bildschirmausgabe . . . 19
6.2 Die Endzeile . . . 19
6.3 Diagnose bei Befehl 164 . . . 20
6.4 Knotenprotokoll und Protokolldatei . . . 20
6.5 Ergebnisprüfung . . . 22

7 Wenn etwas schiefgeht . . . 23

8 Beispiele Schritt für Schritt . . . 27
8.1 Ausgabe auf dem Schreibwerk . . . 27
8.2 Alle Bedientasten in einer Tastenfolge . . . 28
8.3 Zeichen vom Lochstreifen lesen und auf den Stanzer ausgeben . . . 30
8.4 Diagnose, Speicherauszug und Prüfungen . . . 31
8.5 Trommel über mehrere Tests und eigene Trommeldatei . . . 33

9 Kurzreferenz . . . 35

Anhang A: Glossar . . . 36

Anhang B: Grenzen des Interpreters . . . 37

Anhang C: Verweise auf das Regelwerk . . . 37

1 Zu dieser Anleitung

1.1 Was der Interpreter tut

Der Interpreter führt Programme des Rechners C 8205 interpretativ aus. Er läuft den Knotengraphen des Befehlsablaufs (Phasen I bis IV) ab und bildet Register, Flags, Trommelspeicher und Peripherie nach. Alle Zeiten (Wortzeit, Wartezeit auf die Trommel) sind 0; die Trommeldrehung wird nicht nachgebildet. Die Trommel selbst ist eine persistente Textdatei (`virtuelle_Geraete/Trommel/trommel.txt`): Ihr Inhalt bleibt über Tests und Aufrufe hinweg erhalten (Abschnitt 3.7).

Sie benutzen den Interpreter, um

- Testprogramme des C 8205 nachvollziehbar durchzurechnen (auf Wunsch Knoten für Knoten protokolliert),
- das Verhalten des Regelwerks zu prüfen,
- Ein- und Ausgabe (Schreibwerk, Lochstreifenleser, Stanzer, Pult-Tastatur) am Terminal zu erproben.

> **Hinweis Grundsatz „laut scheitern“.** Der Interpreter läuft den Graphen wortlautgetreu ab. Ist etwas unklar oder fehlt eine Festlegung, meldet er das und hält an; er rät nie und „repariert“ nichts stillschweigend. Ein Test, der nicht startet, ist daher kein Defekt, sondern eine Meldung, die Sie lesen sollten.

1.2 Wie die Anleitung aufgebaut ist

Kapitel	Inhalt	Wann lesen?
2 Erste Schritte	Voraussetzungen, Ordner, Trommeldatei erzeugen, erster Lauf	beim ersten Mal
3 Tests schreiben	Trommelabschnitt, Marken, Tastenfolge, Prüfungen, Geräte-Zeilen, Trommel zwischen den Tests	beim Anlegen eines Tests
4 Interpreter starten	Aufruf, Schalter, Wrapper, Rückkehrcodes	bei jedem Lauf
5 Ein- und Ausgabe	Kanäle, Schreibwerk und Ausgabefenster, Leser, Stanzer, Tastatur	bei Zeichen-Ein-/Ausgabe
6 Ergebnisse lesen	Endzeile, Diagnose, Protokoll, Ergebnisprüfung	nach jedem Lauf
7 Wenn etwas schiefgeht	Fehlerverzeichnis mit Ursache, Abhilfe und Rückkehrcode, Beispiele	bei Meldungen
8 Beispiele	Beispiele Schritt für Schritt zum Nachmachen	zum Ausprobieren
9 Kurzreferenz	eine Seite zum Ausdrucken	zum Nachschlagen
Anhang A bis C	Glossar, Grenzen des Interpreters, Verweise auf das Regelwerk	bei Bedarf

1.3 Stand und Neuerungen

Die Anleitung gilt für die Interpreter-Version auf Seite 2. Gegenüber der Anleitung vom 08.10.2026 (damals noch ohne Versionsnummer) sind neu:

- Die **Versionsnummer** des Interpreters (Form `y.xx-zzz`) und der Schalter `--version` (Kapitel 4).
- Das **Ausgabefenster** „SW Ausgabe“ mit den Schaltern `--SW-TERMINAL` und `--zwischenablage`; es zeigt auch das Echo der K0-Eingabe (Kapitel 4 und 5).
- Die Testmarken `#STARTE n` (Makro für den Ansprung eines Speicherplatzes) und `#INTERRUPT H …` (Unterbrechung des Programmlaufs mit weiteren Tasten) (Abschnitt 3.2).
- Die Beispiele laufen bei jeder Erzeugung des PDFs neu; ihre Ausgaben sind immer die der genannten Version.

2 Erste Schritte

2.1 Voraussetzungen

- **Python 3.** Es wird nur die Standardbibliothek benutzt; es ist nichts zu installieren.
- Verzeichnis `markdown-dateien/` neben `c8205_interp.py` (Vorgabe für `--mdverz`) mit `C8205-Interpreter-Eingänge_der_Knoten.md` (Knotengraph) und `C8205-Interpreter-Beschreibung_der_Knoten.md` (Variablentabelle T2). Der Interpreter liest beide bei jedem Start; fehlt eine, endet der Aufruf mit Rückkehrcode 2.

- Die Trommeldatei `virtuelle_Geraete/Trommel/trommel.txt` neben `c8205_interp.py` (Vorgabe für `--trommel`): genau 4096 Zeilen, les- und beschreibbar. Der Interpreter legt sie nie selbst an; Abschnitt 2.3 zeigt, wie Sie sie erzeugen.
- Optional der Wrapper `c8205_interpreter.sh` (Linux/macOS) bzw. `c8205_interpreter.bat` (Windows), ebenfalls im Ordner des Skripts.
- Ein Terminal mit ANSI-Farben (für Dunkelgelb und Rot). Ohne Terminal oder mit gesetztem `NO_COLOR` erscheint reiner Text.
- **Nur für das Ausgabefenster:** `tkinter` (Linux: `sudo apt install python3-tk`; unter Windows und macOS im Python-Installer von python.org enthalten) und eine grafische Oberfläche. Zum Kopieren aus dem Fenster unter Linux optional `xsel` oder `xclip` (X11) bzw. `wl-clipboard` (Wayland); unter Windows ist nichts nachzuinstallieren.

Hinweis Der Dateiname der Knotengraph-Datei enthält ein „ä“. Findet der Interpreter sie nicht (z. B. wegen der Zeichenkodierung eines Archivs), geben Sie das Verzeichnis mit `--mdverz` an.

2.2 Ordner

```
Projektordner/
  c8205_interp.py
  c8205_interpreter.sh     c8205_interpreter.bat
  erzeuge_leere_trommel.py erzeugt bzw. leert die Trommeldatei
  markdown-dateien/        Regelwerk (u. a. Eingänge_der_Knoten.md, Beschreibung_der_Knoten.md)
  Tests/                   Testdateien (.txt)
  Test-Protokolle/         Protokolle (.log), vom Wrapper angelegt
  virtuelle_Geraete/
    Trommel/trommel.txt    die Trommel: immer benutzt (4096 Zeilen)
    Leser1/ Leser2/ Stanzer1/ SW-Ausgabe/ T-SW-Eingabe/
                           Ablage für virtuelle Geräte (NICHT automatisch benutzt)
```

Hinweis Die Ordner `Leser1/`, `Leser2/`, `Stanzer1/`, `SW-Ausgabe/` und `T-SW-Eingabe/` unter `virtuelle_Geraete/` werden nicht automatisch benutzt. Sie wirken nur, wenn Sie den Schalter beim Aufruf angeben (auch beim Wrapper: `./c8205_interpreter.sh Tests/x.txt --leser1 virtuelle_Geraete/Leser1`, Abschnitt 4.3) oder wenn die Testdatei den Pfad als Geräte-Zeile selbst mitbringt (Abschnitt 3.6). Die einzige Datei unter `virtuelle_Geraete/`, die der Interpreter immer benutzt, ist `Trommel/trommel.txt`.

2.3 Die Trommeldatei erzeugen

Der Interpreter braucht bei jedem Aufruf die Trommeldatei, seinen Speicher. Er legt sie nie selbst an und setzt sie nie zurück. Erzeugen Sie sie einmal und immer dann neu, wenn Sie eine leere Trommel brauchen (Abschnitt 3.7):

```
$ python3 erzeuge_leere_trommel.py
$ wc -l virtuelle_Geraete/Trommel/trommel.txt
4096 virtuelle_Geraete/Trommel/trommel.txt
$ head -n 3 virtuelle_Geraete/Trommel/trommel.txt
0000: 00000000000
0001: 00000000000
0002: 00000000000
$ python3 erzeuge_leere_trommel.py --aus trommeln/leer.txt
$ cmp virtuelle_Geraete/Trommel/trommel.txt trommeln/leer.txt && echo gleich
gleich
```

Das Skript schreibt 4096 Zeilen `aaaa: ttttttttttt` mit dem Wert `00000000000` (Adressen 0000 bis 7777, oktal, Zeilenende LF) und überschreibt eine vorhandene Datei ohne Rückfrage. Mit `--aus PFAD` entsteht die Datei an anderer Stelle; fehlende Ordner werden angelegt. Das Skript gibt nichts aus; im Beispiel prüfen `wc` und `head` das Ergebnis.

Achtung Alle Beispiele dieser Anleitung laufen auf einer leeren Trommel. Wer sie nachmacht, erzeugt vor jedem Beispiel eine leere Trommel; sonst hängt das Ergebnis vom Inhalt ab, den frühere Läufe hinterlassen haben (Abschnitt 3.7).

2.4 Der erste Lauf

Legen Sie die Datei `Tests/0015-030.txt` mit folgendem Inhalt an. Das Programm wählt K2 an und gibt auf dem Schreibwerk einen Text aus.

```
Trommeldatei:
#DOK-0015 Test Ausgabebefehl
#PROT STUFE=2
0001: 00007240000
7240: 00004500030
7241: 00002720030
7242: 00000570030
7243: 00001210030
7244: 00001170030
7245: 00000610030
7246: 00000700030
7247: 00001110030
7250: 00000320030
7251: 00000010030
7252: 00000310030
7253: 00000070030
7254: 00000020030
7255: 00000015164
7256: 00000000020

Tastenfolge:
H G M-drücken E M-loslassen R
```

Datei `Tests/0015-030.txt`

Starten Sie den Test mit dem Wrapper:

```
$ ./c8205_interpreter.sh Tests/0015-030.txt
=== Test 0015-030 (#PROT STUFE=2) ===

Jahr 1972
-- Diagnose --
#DOK-0015 Test Ausgabebefehl
Ende: Unbedingter Stop (US:=1) | Knoten: 308 | BZ=7257 AC=00000002000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Protokolldatei: Test-Protokolle/0015-030.log
$ grep -E "^(0001|7240|7256):" virtuelle_Geraete/Trommel/trommel.txt
0001: 00007240000
7240: 00004500030
7256: 00000000020
```

Das Programm hat den Text „Jahr 1972“ ausgegeben (dunkelgelb; die Leerzeile davor ist ein Zeilenumbruch des Programms) und mit einem unbedingten Stopp gehalten. Die Zeile „Ende: …“ fasst den Endzustand zusammen (Kapitel 6). Das ausführliche Protokoll liegt in `Test-Protokolle/0015-030.log`. Die Zellen des Trommelabschnitts stehen jetzt auch in der Trommeldatei, wie die `grep`-Ausgabe zeigt.

> **Achtung** Die Schreibweise der Buchstaben („Jahr“, „JAHR“ oder „jahr“) hängt von der Zeichenlage KB/GB des Schreibwerks ab, die beim Start zufällig gesetzt wird. Dieses Programm gibt selbst Umschaltzeichen aus; ohne solche Zeichen wäre die Groß-/Kleinschreibung zufällig. Auch die „Knoten“-Zahl der Endzeile ist fassungsabhängig und kein fester Sollwert.

3 Tests schreiben

Eine Testdatei besteht aus zwei Teilen, optional drei: Der Teil `Trommeldatei:` (der Trommelabschnitt) nennt die Zellen, die vor dem Test auf die Trommel geschrieben werden, der Teil `Tastenfolge:` die Bedienhandlungen am Pult vor dem Start, der Teil `Prüfungen:` (optional) die erwarteten Ergebnisse.

3.1 Trommelabschnitt

Jede Zeile hat die Form `Adresse: Wert [# Bemerkung]`. Die Adresse hat vier Oktalstellen, der Wert elf. Der Wert besteht aus vier Stellen „beliebig“ und sieben Stellen Befehl (vier Adresse + drei Operation): `00004500030` ist also Adresse `4500`, Operation `030`. Leerzeilen und Zeilen, die mit `#` beginnen, werden übergangen; ein Kommentar darf hinter einer `Adresse: Wert`-Zeile stehen, wenn mindestens ein Leerzeichen davorsteht. Nicht genannte Zellen behalten ihren bisherigen Inhalt; auf einer leeren Trommel sind das 0 (Abschnitt 3.7).

```
# Beginn Programmbefehle
0000: 00003000211
0001: 00003001210
0002: 00000000020
# Beginn Datenbereich
3000: 00000000100 # Startwert
3001: 00000000045
```

Achtung Eine Zelle, die weder der Test noch ein früherer Lauf belegt hat, ist 0, und der Wert 0 ist als Befehl der unbedingte Sprung nach Adresse 0000. Läuft Ihr Programm über sein Ende hinaus, springt es zurück an den Anfang. Schließen Sie jedes Programm mit einem Stopp-Befehl `00000000020` ab. Weil die Trommel zwischen den Tests erhalten bleibt, kann eine „leere“ Zelle auch Reste eines früheren Tests enthalten (Abschnitt 3.7).

Hinweis Wenn Sie in Assemblersprache programmieren, erzeugt der Assembler `c82ass` diese Zeilen für Sie (Datei `<n>.test`); Sie ergänzen nur Testmarken und Tastenfolge. Siehe PDF „Testhandbuch“.

3.2 Testmarken

Testmarken sehen aus wie Kommentare (`#`), haben aber eine Wirkung. Sie gehören nur zum Test; es gibt sie in der Hardware nicht.

Marke	Wirkung
`#DOK-xxxx Text`	Kommentar zum Test (xxxx oktal). Beim Befehl 164 („Leerbefehl“) mit der Adresse xxxx wird der Text als Diagnose ausgegeben, unabhängig von der Protokollstufe. Mehrfach zulässig.
`#SPEI-xxxx ssss-eeee`	Speicherbereich ssss bis eeee (oktal, ssss ≤ eeee), der beim Befehl 164 mit der Adresse xxxx ausgegeben wird. Unabhängig von #DOK: auch einzeln und mehrfach zulässig.
`#SPUR-xxxx aaaa`	Ganze Spur (Bahn), in der aaaa liegt: alle 32 Zellen in physischer Zugriffsreihenfolge der Trommel, ausgegeben beim Befehl 164 mit der Adresse xxxx. Unabhängig von #DOK.
`#PROT STUFE=n`	Protokollstufe des Tests (höchstens einmal): 0 = kein Knotenprotokoll, 1 = Knotenzeilen auf dem Bildschirm, 2 = Knotenzeilen nur in der Protokolldatei. Fehlt die Marke, gilt 1.
`#AC=ooooooooooo`	Anfangswert des Akkumulators, 11 Oktalziffern (höchstens einmal). Fehlt die Marke, startet der AC mit 0.
`#BZ=oooo`	Anfangswert des Befehlszählers, 4 Oktalziffern (höchstens einmal). Fehlt die Marke, startet der BZ mit 0.
`#STARTE n`	Makro für die Tastenfolge `H G M+ E … E M- R` (n Anschläge E): Ansprung des Speicherplatzes n. n hat 1 bis 4 Oktalziffern; höchstens einmal; ersetzt die Tastenfolge.
`#INTERRUPT H <Tasten>`	Unterbricht den Programmlauf nach dem Befehl der davorstehenden Zeile und bedient die angegebenen Tasten. Nur zwischen zwei Befehlszeilen; höchstens einmal je Befehlszeile.

Achtung Die Diagnose erscheint nur, wenn der Test den Befehl 164 mit passender Adresse ausführt. Schreiben Sie ihn an die Stelle, an der Sie auswerten wollen: `00000020164` an Adresse 0002 löst die Marken mit `xxxx = 0020` aus. Der Leerbefehl wird zuerst ausgeführt, die Diagnose erscheint danach. Eine automatische Zustandszeile gibt es nicht.

Anfangswerte `#AC=` und `#BZ=`

Beide Marken stehen im Trommelabschnitt, sind voneinander unabhängig und einzeln oder zusammen zulässig. Sie setzen Akkumulator und Befehlszähler als Anfangszustand vor der ersten Taste. Die Taste `G` wirkt danach unverändert: Sie lässt AC und BZ unberührt, stellt aber das Befehlsregister auf einen Sprung nach 0000. `#BZ=` wirkt daher nur, solange keine Taste `G` davor kommt, z. B. bei der Tastenfolge `H M-drücken E M-loslassen R`. Das erste Beispiel startet mit AC = 100 und addiert die Zelle 3000 (45):

```
Trommeldatei:
#PROT STUFE=0
#AC=00000000100
0000: 00003000210 # AC := AC + <3000>
0001: 00000000020
3000: 00000000045
Tastenfolge:
H G R
```

Datei `Tests/ac-start.txt`

```
$ python3 c8205_interp.py Tests/ac-start.txt; echo "Rückkehrcode: $?"
=== Test ac-start (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Das zweite Beispiel hat keine Taste `G`: `#BZ=0010` setzt den Befehlszähler, das `E` unter gehaltenem `M` zählt auf 0011 und der Ablauf beginnt dort:

```
Trommeldatei:
#PROT STUFE=0
#BZ=0010
0011: 00000000020
Tastenfolge:
H M-drücken E M-loslassen R
```

Datei `Tests/bz-start.txt`

```
$ python3 c8205_interp.py Tests/bz-start.txt; echo "Rückkehrcode: $?"
=== Test bz-start (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0012 AC=00000012000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Ein ungültiger Wert (falsche Stellenzahl, Ziffer 8 oder 9, leerer Wert) oder eine doppelte Marke ist ein Fehler der formalen Prüfung (Abschnitt 3.5). Beispiel: In `Tests/ac-doppelt.txt` steht `#AC=` zweimal.

```
Trommeldatei:
#PROT STUFE=0
#AC=00000000100
#AC=00000000200
0000: 00000000020
Tastenfolge:
H G R
```

Datei `Tests/ac-doppelt.txt`

```
$ python3 c8205_interp.py Tests/ac-doppelt.txt; echo "Rückkehrcode: $?"
=== Test ac-doppelt (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 4: #AC mehrfach
Rückkehrcode: 16
```

Das Makro `#STARTE n`

`#STARTE n` ersetzt die Tastenfolge zum Ansprung des Speicherplatzes `n` (Muster aus Abschnitt 3.3). `n` ist eine Oktalzahl mit 1 bis 4 Ziffern und zugleich die Zahl der Anschläge `E`: `#STARTE 12` löst 10 (dezimal) Anschläge aus und spricht den Speicherplatz `0012` an. Die Marke steht an beliebiger Stelle außer im Teil `Prüfungen:`, üblicherweise bei den Testmarken; der Teil `Tastenfolge:` darf dann fehlen. Enthält er Tasten, ist das ein Formfehler, ebenso ein fehlender oder ungültiger Wert und eine doppelte Marke. `#AC=`, `#BZ=` und `#INTERRUPT` sind mit dem Makro verträglich. Im Protokoll erscheinen lange `E`-Folgen unter gehaltenem `M` gekürzt; die Rechnung ist davon nicht berührt.

```
Trommeldatei:
#PROT STUFE=0
#STARTE 4
0004: 00003000211 # 211 Laden
0005: 00003001210 # 210 Addieren
0006: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Prüfungen:
BZ=0007 AC=00000000145 US=1
```

Datei `Tests/start-4.txt`

```
$ python3 c8205_interp.py Tests/start-4.txt; echo "Rückkehrcode: $?"
=== Test start-4 (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 66 | BZ=0007 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0007 ist=0007 OK
  AC: soll=00000000145 ist=00000000145 OK
  US: soll=1 ist=1 OK
Test wahr
Rückkehrcode: 0
```

> **Achtung** `#STARTE 0` ergibt die Folge `H G M+ M- R`, also ein Antippen von `M` ohne `E`. Das ist kein schlichter Start bei 0000, sondern ein aufgestellter Sprung mit Rückkehrabsicht nach 0000. Für einen Start bei 0000 dient `H G R`.

Die Marke `#INTERRUPT`

`#INTERRUPT` bildet einen Eingriff des Bedieners mitten im Programm nach. Die Zeile steht im Trommelabschnitt **zwischen zwei Befehlszeilen** und gehört zu der Zeile davor (Adresse `p`). Nach dem Befehl auf `p` wird der Lauf unterbrochen, aber nur, wenn er sequentiell weiterläuft (BZ = `p+1`); springt der Befehl woandershin, bleibt die Zeile ohne Wirkung. Die erste Taste muss `H` sein; danach folgen beliebige zulässige Tasten, eine gehaltene Taste wird mit `+`, ihr Loslassen mit `-` geschrieben. Endet die Folge mit `R` oder `ST`, läuft das Programm weiter; jede andere letzte Taste beendet den Test an dieser Stelle im Halt, und `Prüfungen:` gilt für diesen Zustand. Die Zeile ist keine Speicherzelle und kein Sprungziel. Im ersten Beispiel läuft das Programm nach der Unterbrechung weiter:

```
Trommeldatei:
#PROT STUFE=0
0000: 00003000211 # 211 Laden
#INTERRUPT H R
0001: 00003001210 # 210 Addieren
0002: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045
Tastenfolge:
H G R
Prüfungen:
BZ=0003 AC=00000000145 US=1
```

Datei `Tests/int-weiter.txt`

```
$ python3 c8205_interp.py Tests/int-weiter.txt; echo "Rückkehrcode: $?"
=== Test int-weiter (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 68 | BZ=0003 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0003 ist=0003 OK
  AC: soll=00000000145 ist=00000000145 OK
  US: soll=1 ist=1 OK
Test wahr
Rückkehrcode: 0
```

Im zweiten Beispiel transportiert die Unterbrechung den AC nach Zelle 7777 (`H G Ü E`); die letzte Taste ist `E`, der Test endet daher im Halt, und die Prüfung gilt für den Zustand danach:

```
Trommeldatei:
#PROT STUFE=0
0000: 00003000211 # 211 Laden
#INTERRUPT H G Ü E # Transport AC nach 7777, Test endet (letzte Taste E)
0001: 00003001210 # 210 Addieren
0002: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045
Tastenfolge:
H G R
Prüfungen:
7777: 00000000100
```

Datei `Tests/int-transport.txt`

```
$ python3 c8205_interp.py Tests/int-transport.txt; echo "Rückkehrcode: $?"
=== Test int-transport (#PROT STUFE=0) ===
Ende: Halt (Schalter H): INTERRUPT nach Befehl 0000 endet ohne R/ST | Knoten: 27 | BZ=0002 AC=00000000100 AR=3001
    | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
Ergebnispruefung:
  7777: soll=00000000100 ist=00000000100 OK
Test wahr
Rückkehrcode: 0
```

Achtung Ohne vorheriges `G` ist das OR der Operationsteil des letzten Befehls: `H Ü E` ergibt nach einem Ladebefehl 211 die Operation 251, nicht 240. Für den reinen Transport nach 7777 ist `H G Ü E` nötig. Formfehler (Beispiel in Kapitel 7): `#INTERRUPT` vor der ersten oder hinter der letzten Befehlszeile, ohne Tasten, mit erster Taste außer `H`, mit unzulässiger M-Folge, zweimal hinter derselben Befehlszeile.

3.3 Tastenfolge

Die Tastenfolge (hinter `Tastenfolge:`) legt den Ausgangszustand fest. Die Token stehen durch Leerzeichen getrennt in einer oder mehreren Zeilen; Groß-/Kleinschreibung ist gleichgültig. Sie endet mit `R` (dann ist `H` am Ende gelöst) oder im Halt (`H` bleibt gesetzt, z. B. `H G Ü E`). Beides ist gültig. Endet sie weder mit `R` noch im Halt, startet der Test nicht.

Token	Wirkung
`H`	Schalter H-R auf „H“: Flag H := 1
`G`	Generallöschen: alle Flags außer H, T, BS, SP := 0, danach US := 1; AR := 0000, OR := 000 (Befehlsregister 0000 000: unbedingter Sprung nach 0000); Kanal K0 angewählt (T := 1, bei gedrücktem SW: SW := 1). **AC und BZ bleiben unverändert**; BZ := 0 entsteht erst, wenn `R` oder `E` (ohne `M`) den Sprung ausführt.
`M+`, `M-drücken`, `M-gedrückt-halten`	M gedrückt halten: OR := OR ODER 025
`M-`, `M-loslassen`	M loslassen: der Beitrag 025 im OR entfällt
`M`	Antippen = `M+` und sofort `M-`: OR := OR ODER 025, BR bleibt aufgestellt. `R` startet bei Knoten 54, wenn BR ein Sprung mit Rückkehrabsicht ist (025, 027, 125, 127), sonst bei Knoten 19.
`E` (mit gehaltenem M)	BZ := BZ + 1, AR := BZ, AC := BZ+1 im Adressfeld; Befehlsentnahme gesperrt
`E` (ohne M)	nur bei H: Einzelschritt über den Knotengraphen ab Knoten 19 (nach `M` mit Sprung mit Rückkehrabsicht ab Knoten 54), Halt vor erneutem Eintritt in Knoten 19

Token	Wirkung
`BS=1`, `SP=1` (auch `BS+`, `SP+`)	Schalter gedrückt und eingerastet: Flag BS bzw. SP := 1
`BS=0`, `SP=0` (auch `BS-`, `SP-`)	Schalter gelöst: Flag := 0. `G` löst die Schalter nicht.
`ST`	nicht an H gebunden, nur bei US oder BS wirksam (sonst keine Wirkung): OR := OR ODER 240, AR bleibt, US := 0; BR aufgestellt. Nach einem Stopp-Befehl ist das die Operation 26Z, nach `G` die Operation 240.
`Ü` (auch `UE`)	nur bei H: AR := 7777, OR := OR ODER 240; BR aufgestellt
`SW=1`, `SW+`, `SW-drücken`	Schalter SW gedrückt: Flag SW := 1, T := 0

Token	Wirkung
`SW=0`, `SW-`, `SW-loslassen`	Schalter SW gelöst: Flag SW := 0, T := 1
`K0`, `K1`, `K2`, `K3`	Kanaltaste (rastend): Flag des gewählten Kanals := 1, die der anderen := 0; bei `K0` zusätzlich T oder SW je nach Schalter SW. Ein im Programm ausgeführter Anwahlbefehl übersteuert die Einstellung.
`R`	Schalter H-R auf „R“: H := 0, US := 0; Ablauf beginnt bei Konnektor B (bei aufgestelltem BR bei Knoten 19, nach `M` mit Sprung mit Rückkehrabsicht bei Knoten 54)

> **Achtung** `SW=1` muss vor `G` stehen, weil `G` daraus für K0 T oder SW ableitet. Die Kanaltasten `K0` … `K3` stehen nach `G`, weil `G` den Kanal K0 wieder anwählt. Nach `Ü`, `ST` oder `E` (ohne M) ist BR aufgestellt: folgt `R`, beginnt der Ablauf bei Knoten 19 statt bei Konnektor B.

Ungültig (der Test startet nicht): `M-` ohne `M+`, `M+` bei gedrücktem M, `R` oder Ende bei gedrücktem M, unbekannte Token. Die Eingabetasten (0…9, `+` usw.) gehören nicht in die Tastenfolge; Eingaben an K0 stehen in Abschnitt 5.4.

Muster: Sprung nach Speicherplatz n. Mit `H G M+ E … E M- R` (n-mal `E`) beginnt der Ablauf bei Adresse n; kürzer schreibt man `#STARTE n` (Abschnitt 3.2). Beispiel n = 4, das Programm steht ab Adresse 0004:

```
Trommeldatei:
#PROT STUFE=0
0004: 00000000020
Tastenfolge:
H G M+ E E E E M- R
```

Datei `Tests/sprung-n.txt`

```
$ python3 c8205_interp.py Tests/sprung-n.txt; echo "Rückkehrcode: $?"
=== Test sprung-n (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0005 AC=00000005000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Der Ablauf hat bei Adresse 0004 begonnen und dort gehalten (BZ=0005). Der AC enthält – wie bei der Taste `M` üblich – im Adressfeld die um 1 erhöhte Adresse (`…05000`). Rückkehrcode 4: Der Test enthält keine Prüfung (Abschnitt 4.4).

Tastenfolge im Halt. Endet die Tastenfolge ohne `R`, startet der Rechner nicht; der Lauf endet sofort mit dem Grund „Halt (Schalter H)“ und dem Rückkehrcode 4. Beispiel: Tastenfolge `H G`.

```
Trommeldatei:
#PROT STUFE=0
0000: 00000000020
Tastenfolge:
H G
```

Datei `Tests/halt-ende.txt`

```
$ python3 c8205_interp.py Tests/halt-ende.txt; echo "Rückkehrcode: $?"
=== Test halt-ende (#PROT STUFE=0) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0
    US=1 E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

Auch `H G Ü E` endet im Halt: `Ü` stellt den Transport-Befehl auf, `E` (ohne `M`) führt genau einen Schritt aus (Einzelschritt) und hält wieder.

```
$ python3 c8205_interp.py Tests/ue-e.txt; echo "Rückkehrcode: $?"
=== Test ue-e (#PROT STUFE=0) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0
    US=1 E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

3.4 Prüfungen (optional)

Hinter der Tastenfolge kann ein Teil `Prüfungen:` stehen. Er enthält höchstens eine Zeile mit Zuweisungen `SCHLÜSSEL=WERT` für Register und Flags (`BZ`, `AC`, `AR`, `FE`, `FUE`, `BS`, `US`, `E`, `A`, `V1`, `V2`; `BZ` und `AR` vierstellig, `AC` elfstellig, Flags 0 oder 1, jeder Schlüssel höchstens einmal), beliebig viele Speicherzeilen `Adresse: Wert` und bis zu zwei Dateiprüfungen. Nicht genannte Schlüssel werden nicht geprüft.

```
Prüfungen:
BZ=0004 AC=00000000145 US=1
3000: 000000000100
--pruefe-stanzer1 soll.txt
```

Der Interpreter vergleicht nach dem Lauf jeden Wert einzeln und meldet `Test wahr` oder `Test falsch` (Beispiele in Abschnitt 6.5). `--pruefe-stanzer1 DATEI` und `--pruefe-sw-ausgabe DATEI` vergleichen die vom Lauf erzeugte Stanzer- bzw. Schreibwerk-Datei mit der Soll-Datei `DATEI`; ein Name ohne Ordner liegt neben der Testdatei. Jede Zeile steht höchstens einmal; der Schalter gleichen Namens auf der Kommandozeile hat Vorrang. Die Dateiprüfung setzt das zugehörige Gerät voraus (`--stanzer1` bzw. `--sw-ausgabe`, auch als Geräte-Zeile). Die Speicherzeilen lesen den aktuellen Trommelstand, auch mit Resten früherer Tests: Prüfen Sie nur Zellen, die der Test selbst nennt, oder starten Sie auf leerer Trommel (Abschnitt 3.7). Dateiprüfung ohne Gerät ist ein formaler Fehler:

```
$ python3 c8205_interp.py Tests/pruefe-ohne.txt; echo "Rückkehrcode: $?"
=== Test pruefe-ohne (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - --pruefe-stanzer1 ohne --stanzer1: es gibt keine erzeugte Datei, die verglichen werden koennte
Rückkehrcode: 16
```

Hinweis Bricht der Lauf vorher ab (Stub, Gerätefehler), wird nicht geprüft; die Ausgabe lautet dann „Ergebnisprüfung: nicht durchgeführt“, und der Test gilt nicht automatisch als „falsch“. Ein leerer Abschnitt `Prüfungen:` gilt als fehlender.

Achtung Die Quelle „Durchführung eines Tests“ beschreibt die Kopfzeile `Prüfungen: False` (Negativtests, die `Test falsch` ergeben sollen). Diese Fassung des Interpreters kennt sie noch nicht: Der Test startet nicht, sondern endet mit Rückkehrcode 16. Schreiben Sie bis dahin `Prüfungen:` ohne Zusatz und rechnen Sie mit `Test falsch` (Rückkehrcode 6).

```
$ python3 c8205_interp.py Tests/pruef-false.txt; echo "Rückkehrcode: $?"
=== Test pruef-false (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 8: Pruefungen: ungueltiges Element 'False'
  - Zeile 9: Pruefungen: ungueltige Zeile: 'AC=00000000101'
Rückkehrcode: 16
```

3.5 Formale Prüfung vor dem Start

Vor dem Start prüft der Interpreter: Zeilenformat, keine Adresse mit widersprüchlichem Wert, gültige `#SPEI`-Spanne, gültige `#PROT`-Stufe, gültige Marken `#AC=` und `#BZ=` (je höchstens einmal), gültige `#INTERRUPT`-Zeilen, gültiges `#STARTE`, vorhandene und gültige Tastenfolge (bei `#STARTE` entfällt sie), gültiger `Prüfungen:`-Teil sowie die angegebenen Geräte-Pfade (`--leser1`, `--leser2`, `--k0 datei=`, auch als Zeile der Testdatei). Bei einer Unstimmigkeit startet der Test nicht (Rückkehrcode 16) und wird nicht eigenmächtig korrigiert. Die Trommeldatei wird nicht je Test, sondern einmal vor dem ersten Test des Aufrufs geprüft (Abschnitt 3.7).

```
Trommeldatei:
#PROT STUFE=0
0000: 000000002
0001: 00000000020
0001: 00000000010
Tastenfolge:
H G R
```

Datei `Tests/formal.txt`

```
$ python3 c8205_interp.py Tests/formal.txt; echo "Rückkehrcode: $?"
=== Test formal (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: ungueltiges Zeilenformat: '0000: 000000002'
  - Zeile 5: Adresse 0001 widerspruechlich belegt
Rückkehrcode: 16
```

Zeile 3 hat nur neun statt elf Stellen im Wert; Adresse 0001 kommt zweimal mit verschiedenen Werten vor.

3.6 Geräte in der Testdatei

Der Test kann seine Geräte selbst festlegen. Dazu steht je eine Zeile in der Testdatei, an beliebiger Stelle, üblicherweise bei den Testmarken im Trommelabschnitt. Die Zeilen haben dieselbe Form wie die gleichnamigen Schalter:

```
--leser1 PFAD                # Lochstreifen für Leser 1 (K1): Datei oder Ordner
--leser2 PFAD                # Lochstreifen für Leser 2 (K2): Datei oder Ordner
--stanzer1 PFAD              # Ausgabedatei bzw. -ordner des Stanzers 1 (K1)
--sw-ausgabe PFAD            # Schreibwerk-Ausgabe (K2) nach PFAD/sw-ausgabe.txt
--k0 datei=PFAD              # K0-Eingabe (T und SW) aus Datei oder Ordner
```

- Jede Zeile darf höchstens einmal vorkommen; ein Kommentar `# …` am Zeilenende ist zulässig.
- Die Kommandozeile hat Vorrang: Ist derselbe Schalter beim Aufruf angegeben, gilt dessen Wert; die Zeile der Testdatei wird übergangen.
- Ein relativer Pfad gilt vom aktuellen Ordner des Aufrufs aus (wie auf der Kommandozeile), nicht vom Ordner der Testdatei.
- Eine Zeile ohne Wert, eine doppelte Zeile, `--k0` ohne `datei=` und ein nicht vorhandener Leser- oder `--k0`-Pfad sind Formfehler: Der Test startet nicht (Rückkehrcode 16; Beispiel in Kapitel 7).
- Zeigt `--leser1` oder `--leser2` auf einen Ordner, erscheint ein Hinweis: Alle Dateien darin werden als Lochstreifen gelesen; für einen Test ist eine einzelne Datei sicherer.
- `--SW-TERMINAL` ist **keine** Geräte-Zeile: Der Schalter gilt nur auf der Kommandozeile; als Zeile der Testdatei ist er ein Formfehler (Rückkehrcode 16).
- Mit der Zeile `--sw-ausgabe SW-Ausgabe` braucht der Aufruf keinen Schalter; am Terminal erscheint nichts, die Zeichen stehen in `SW-Ausgabe/sw-ausgabe.txt`.

Beispiel: Die Testdatei `sw-zeile.txt` wählt K2 an, stellt mit `Um. KB` (Ausgabe-Adresse `117x`) die Kleinbuchstabenlage ein, damit die Ausgabe nicht vom Zufall abhängt, und gibt die Zeichen 5 und Punkt aus; die Zeile `--sw-ausgabe SW-Ausgabe` steht bei den Testmarken. Das Terminal zeigt nur die Endzeile, die Zeichen stehen in der Datei:

```
Trommeldatei:
--sw-ausgabe SW-Ausgabe # Schreibwerk-Ausgabe in Datei
#PROT STUFE=0
0000: 00004500030
0001: 00001170030
0002: 00000250030
0003: 00000730030
0004: 00000000020
Tastenfolge:
H G R
```

Datei `Tests/sw-zeile.txt`

```
$ python3 c8205_interp.py Tests/sw-zeile.txt; echo "Rückkehrcode: $?"
=== Test sw-zeile (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 93 | BZ=0005 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
$ cat SW-Ausgabe/sw-ausgabe.txt
5.
```

3.7 Die Trommel zwischen den Tests

Die Trommel ist keine Datei je Test, sondern eine dauerhafte Textdatei (Vorgabe `virtuelle_Geraete/Trommel/trommel.txt`, Schalter `--trommel`). Sie hält ihren Inhalt über Tests und Aufrufe hinweg. Der Trommelabschnitt einer Testdatei ist davon zu unterscheiden: Er nennt nur die Zellen, die der Test vor dem Start auf die Trommel schreibt.

- **Format:** genau 4096 Zeilen `aaaa: ttttttttttt` (vier Oktalstellen Adresse, Doppelpunkt, Leerzeichen, elf Oktalstellen Wert, Zeilenende LF) in aufsteigender Adresse 0000 bis 7777, jede Adresse genau einmal; keine Kommentar- und keine Leerzeilen.
- **Prüfung:** Die Trommeldatei wird einmal je Aufruf vor dem ersten Test geprüft. Ist sie fehlerhaft oder nicht vorhanden, startet kein Test des Aufrufs (Rückkehrcode 16, Kapitel 7).
- **Beginn eines Tests:** Der Interpreter schreibt die Zeilen des Trommelabschnitts auf die Trommel; sie überlagern die vorhandenen Werte. Zellen, die der Test nicht nennt, behalten ihren bisherigen Inhalt. Register, Flags und Geräte beginnen dagegen bei jedem Test neu.

- **Während des Tests:** Jeder Schreibzugriff eines Befehls ändert die Zelle auch in der Datei. Nach dem Test bleibt der Endzustand stehen; er ist der Anfangszustand des nächsten Tests, auch wenn ein Aufruf mehrere Testdateien nacheinander abarbeitet.
- **Zurücksetzen:** Der Interpreter setzt die Trommel nie zurück und legt die Datei nie an. Eine leere Trommel erzeugt `python3 erzeuge_leere_trommel.py` (Abschnitt 2.3).

Achtung Ein Test, der eine Zelle liest oder prüft, die er im Trommelabschnitt nicht selbst nennt, erbt deren Inhalt von den vorigen Läufen. Das gilt auch für Prüfungen mit Sollwert 0 auf einer nicht belegten Zelle. Empfehlung: Nennen Sie alle Zellen, auf die sich das Ergebnis stützt, auch mit Wert 0 (`3000: 00000000000`), oder erzeugen Sie vor dem Lauf eine leere Trommel. Dann ist der Test von der Vorgeschichte unabhängig.

Das vollständige Beispiel mit zwei Tests, die dieselbe Zelle 3000 benutzen, und mit dem Schalter `--trommel` steht in Abschnitt 8.5.

Hinweis Die Reihenfolge der Tests bestimmt den Anfangszustand jedes Tests. Sammelläufe über viele Testdateien liefern nur dann reproduzierbare Ergebnisse, wenn jeder Test seine Zellen selbst nennt oder die Reihenfolge feststeht. Sollwerte aus Regressionsläufen gelten für einen Lauf auf leerer Trommel.

Hinweis Die Trommeldatei muss LF-Zeilenenden haben (jede Zeile 18 Byte). Eine Datei mit CRLF (z. B. unter Windows im Textmodus erzeugt oder durch Git umgewandelt) erkennt der Interpreter beim Start und weist sie ab (Rückkehrcode 16, Kapitel 7); `erzeuge_leere_trommel.py` schreibt immer LF.

4 Interpreter starten

4.1 Kommandozeile

```
python3 c8205_interp.py [--mdverz VERZ] [--trommel PFAD] [--out VERZ|DATEI.log]
        [--leser1 PFAD] [--leser2 PFAD] [--stanzer1 PFAD] [--sw-ausgabe PFAD]
        [--k0 datei=PFAD] [--knotentest] [--NO-TERMOUT] [--SW-TERMINAL]
        [--zwischenablage auto|tk|system]
        [--pruefe-stanzer1 DATEI] [--pruefe-sw-ausgabe DATEI]  TEST.txt [TEST2.txt ...]
python3 c8205_interp.py --version
```

Schalter	Wirkung	Vorgabe
`TEST.txt`	Testdatei(en). Ist der Pfad nicht vorhanden, wird der Dateiname unterhalb des aktuellen Ordners und des Skript-Ordners gesucht (Groß-/Kleinschreibung egal). Bei mehreren Treffern werden sie aufgelistet und der Test startet nicht.	–
`--mdverz VERZ`	Verzeichnis mit den beiden Regelwerk-Dateien `C8205-Interpreter-Eingänge_der_Knoten.md` (Knotengraph) und `C8205-Interpreter-Beschreibung_der_Knoten.md` (Variablentabelle T2); fehlt eine, bricht der Start ab (Rückkehrcode 2).	`markdown-dateien/` neben dem Skript
`--trommel PFAD`	Die Trommeldatei (4096 Zeilen `aaaa: ttttttttttt`, LF), der persistente Speicher. Sie wird einmal vor dem ersten Test geprüft; ist sie fehlerhaft oder fehlt sie, startet kein Test (Rückkehrcode 16). Nur auf der Kommandozeile, nicht als Zeile der Testdatei. Standardpfad: `virtuelle_Geraete/Trommel/trommel.txt` neben dem Skript.	Standard-Trommel
`--out …`	Mit Dateiendung (z. B. `Test-Protokolle/t1.log`): Protokoll genau in diese Datei (Ordner werden angelegt; nur mit einem Test). Ohne Endung: Ordner, Protokoll dann in `ORDNER/<Testname>/protokoll.txt`.	`Test-Protokolle/`
`--leser1 PFAD`, `--leser2 PFAD`	Lochstreifen (Datei oder Ordner) für Leser 1 (Kanal K1) bzw. Leser 2 (K2). Auch als Zeile der Testdatei möglich (Abschnitt 3.6).	nicht angegeben
`--stanzer1 PFAD`	Ausgabedatei des Stanzers 1 (Kanal K1); ein Ordner (auch ein Name mit „/“ am Ende) ergibt `stanzer-ausgabe.txt` darin. Auch als Zeile der Testdatei möglich.	nicht angegeben
`--sw-ausgabe PFAD`	Schreibwerk-Ausgabe (Kanal K2) in die Datei `sw-ausgabe.txt` dieses Verzeichnisses (ohne Endung: Verzeichnis, wird angelegt). Angelegt erst bei der ersten Ausgabe eines Tests, danach angehängt. Auch als Zeile der Testdatei möglich.	Ausgabe ans Terminal

Schalter	Wirkung	Vorgabe
`--k0 datei=PFAD`	Eingabequelle für Kanal K0 (T und SW): Datei oder Ordner mit einem Zeichen oder Namen je Zeile. Auch als Zeile der Testdatei möglich.	Eingabe vom Terminal
`--pruefe-stanzer1 DATEI`	Dateiprüfung: vergleicht die über `--stanzer1` erzeugte Datei mit der Soll-Datei `DATEI` (ein Name ohne Ordner liegt neben der Testdatei). Setzt `--stanzer1` voraus; nur mit einer Testdatei. Auch als Zeile im Teil `Prüfungen:` möglich (Abschnitt 3.4).	keine Prüfung
`--pruefe-sw-ausgabe DATEI`	wie `--pruefe-stanzer1`, aber für die über `--sw-ausgabe` erzeugte Datei `sw-ausgabe.txt`; setzt `--sw-ausgabe` voraus.	keine Prüfung
`--knotentest`	Nimmt die Zeilen der durchlaufenen Knoten (`Kxxx …`) in das Knotenprotokoll auf. Ändert weder den Ablauf noch die „Knoten“-Zahl der Endzeile.	aus
`--NO-TERMOUT`	Unterdrückt alle Ausgaben des Interpreters selbst am Terminal (Ausnahmen: vom C8205-Programm erzeugte Zeichen, Meldungen bei fehlgeschlagener Vorab-Prüfung und zur Trommeldatei). Der Schalter ist case-sensitiv, genau so zu schreiben.	aus
`--SW-TERMINAL`	Öffnet neben dem Terminal das Textfenster „SW Ausgabe“ (130 × 36 Zeichen, verschiebbar) und gibt jede Schreibwerk-Ausgabe (K2) **zusätzlich** dort aus, gleichgültig ob `--sw-ausgabe` gesetzt ist; außerdem erscheint jede angenommene K0-Eingabe als Echo in Dunkelgelb (Abschnitt 5.4). Braucht `tkinter` und eine grafische Oberfläche; fehlt eines davon, erscheint eine Warnung und der Test läuft ohne Fenster. Nur auf der Kommandozeile; als Zeile der Testdatei ein Formfehler.	aus (kein Fenster)
`--zwischenablage auto|tk|system`	Kopierverfahren des Fensters „SW Ausgabe“ (nur mit `--SW-TERMINAL`): `auto` versucht zuerst ein Systemprogramm bzw. die Win32-API und nur bei Misserfolg Tk; `tk` nutzt nur die Tk-Zwischenablage; `system` nur Systemprogramm bzw. Win32-API. Gedacht für Umgebungen mit eigenem Zwischenablage-Abgleich (z. B. virtuelle Maschinen).	`auto`
`--version`	Gibt die Versionsnummer des Interpreters (Form `y.xx-zzz`) aus und endet mit Rückkehrcode 0, ohne einen Test auszuführen; eine Testdatei ist nicht anzugeben.	–
`-h, --help`	Hilfetext mit allen Schaltern.	–

Hinweis Fenster „SW Ausgabe“ (`--SW-TERMINAL`)**.** Das Fenster bleibt **immer offen**, auch nach dem Ende des Tests, bis Sie es mit Klick auf X schließen. Es läuft als eigener Prozess; jeder Aufruf öffnet ein neues Fenster. Text im Fenster lässt sich mit der Maus markieren und kopieren: sofort beim Loslassen der Maustaste, mit Strg+C oder Strg+Einfg oder über das Kontextmenü (rechte Maustaste). Das gilt unter Linux und unter Windows.

Die langen Bezeichnungen `--lochbandleser1`, `--lochbandleser2` und `--lochbandstanzer1` sind gleichwertig. Ein nicht angegebenes Gerät ist kein Fehler, solange das Programm es nicht benutzt. Benutzt es das Programm doch, erscheint eine rote Fehlermeldung und der Test endet (Kapitel 7). Ausnahmen: K0-Eingabe (Terminal) und Schreibwerk-Ausgabe (Terminal) haben eine Vorgabe und erzeugen auch ohne Schalter keinen Fehler.

Version abfragen. `--version` braucht keine Testdatei und führt keinen Test aus. Die Wrapper sind für Testläufe gedacht; die Version fragen Sie direkt mit `python3` ab. Die Nummer ist fassungsabhängig und kein fester Sollwert.

```
$ python3 c8205_interp.py --version; echo "Rückkehrcode: $?"
c8205_interp.py 0.03-009
Rückkehrcode: 0
```

Fehlerhafte Aufrufe zeigen die Kurzhilfe und enden mit Rückkehrcode 2:

```
$ python3 c8205_interp.py --out ergebnis.log Tests/a.txt Tests/b.txt; echo "Rückkehrcode: $?"
usage: c8205_interp.py [-h] [--version] [--mdverz VERZ] [--trommel PFAD]
                       [--leser1 PFAD] [--leser2 PFAD] [--stanzer1 PFAD]
                       [--sw-ausgabe PFAD] [--pruefe-stanzer1 DATEI]
                       [--pruefe-sw-ausgabe DATEI] [--k0 datei=PFAD]
                       [--out OUT] [--knotentest] [--NO-TERMOUT]
                       [--SW-TERMINAL] [--zwischenablage auto|tk|system]
                       tests [tests ...]
c8205_interp.py: error: --out mit Dateiendung (ergebnis.log) ist nur mit genau einer Testdatei zulaessig
Rückkehrcode: 2
```

Liegen die Regelwerk-Dateien nicht in `markdown-dateien/`, geben Sie das Verzeichnis mit `--mdverz ./markdown-dateien` an; der Wrapper tut das schon (Abschnitt 4.3).

4.2 Mehrere Tests, Protokollort

- Mehrere Testdateien hintereinander: `python3 c8205_interp.py Tests/a.txt Tests/b.txt`. Dann nur `--out` ohne Dateiendung benutzen. Die Tests laufen nacheinander auf derselben Trommel (Abschnitt 3.7).
- `--out Test-Protokolle/t1.log` mit mehreren Tests wird abgelehnt (Beispiel oben, Rückkehrcode 2). Dasselbe gilt für `--pruefe-stanzer1` und `--pruefe-sw-ausgabe`.
- Eine Protokolldatei entsteht nur bei `#PROT STUFE=2`; bei Stufe 0 oder 1 erscheint bei angegebenem `--out` ein Hinweis.

```
$ python3 c8205_interp.py --out ergebnis Tests/kurz2.txt
=== Test kurz2 (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Protokolldatei: ergebnis/kurz2/protokoll.txt
$ ls ergebnis/kurz2
protokoll.txt
```

4.3 Wrapper

Der Wrapper nimmt eine Testdatei entgegen, leitet sie an den Interpreter und schreibt das Protokoll nach `Test-Protokolle/<Testname>.log` (der Name der Testdatei mit der Endung `.log`). Alle weiteren Argumente gibt er unverändert an den Interpreter weiter, sodass jeder Schalter aus Abschnitt 4.1 auch über den Wrapper nutzbar ist. Ohne Testdatei meldet er „Test-Datei fehlt.“ (Code 1). Die Testdatei darf an beliebiger Stelle stehen; in dieser Anleitung steht sie der Übersicht halber zuerst.

```
$ ./c8205_interpreter.sh Tests/0015-030.txt                              # Linux/macOS
> c8205_interpreter.bat Tests\0015-030.txt                               # Windows

# mit zusätzlichen Schaltern:
$ ./c8205_interpreter.sh Tests/x.txt --leser1 virtuelle_Geraete/Leser1
$ ./c8205_interpreter.sh Tests/x.txt --leser1 band5.txt --knotentest
$ ./c8205_interpreter.sh Tests/x.txt --trommel andere/trommel.txt
$ ./c8205_interpreter.sh Tests/x.txt --mdverz ./markdown-dateien

# Kern des Linux-Wrappers:
python3 c8205_interp.py --mdverz ./markdown-dateien --out "Test-Protokolle/$prot_file" "$file" "${rest[@]}"
```

Hinweis Die Vorgaben des Wrappers (`--mdverz`, `--out`) stehen vor den weitergereichten Schaltern; bei doppelter Angabe gilt der zuletzt genannte, ein eigenes `--out` überschreibt also die Vorgabe. Die Wrapper erkennen Schalter mit eigenem Wertargument (`--out`, `--trommel`, `--stanzer1`, `--sw-ausgabe`, `--pruefe-stanzer1`, `--pruefe-sw-ausgabe`, `--zwischenablage` u. a.); der Wert gilt nie als Testdatei. Unter Windows (`.bat`) müssen Werte mit Gleichheitszeichen in Anführungszeichen stehen, z. B. `--k0 "datei=band\k0"`; Pfade mit Leerzeichen ebenfalls. Der Wrapper setzt die Trommel nicht zurück und übergibt `--trommel` nicht. Es wird eine Testdatei je Aufruf verarbeitet.

4.4 Rückkehrcodes

Der Interpreter liefert dem Betriebssystem einen Rückkehrcode, der sich stets auf **genau einen** Test bezieht; bei mehreren Testdateien in einem Aufruf zählt der zuletzt bearbeitete. Der Wrapper gibt den Code unverändert weiter. Ob ein Test „wahr“ oder „falsch“ war, lässt sich so ohne Auswertung der Bildschirmzeilen prüfen.

Code	Bedeutung
0	Prüfung definiert (Teil `Prüfungen:` und/oder Dateiprüfung) und Ergebnis `Test wahr`.
4	Lauf ordnungsgemäß beendet, aber keine Prüfung angegeben (weder `Prüfungen:` noch `--pruefe-…`).
6	Prüfung definiert, Ergebnis `Test falsch` (auch: Soll- oder Ist-Datei fehlt, Dateiinhalt verschieden).
8	Sonstige Laufzeitfehler: Stub, Gerät nicht nutzbar, Knotenlimit, Abbruch mit Strg+C.
16	Vorab-Fehler, der Test startet nicht: formale Prüfung negativ (u. a. fehlender Geräte-Pfad, `--pruefe-…` ohne Gerät, `--SW-TERMINAL` als Zeile der Testdatei), Trommeldatei fehlt oder ist fehlerhaft, ungültige Tastenfolge, Testdatei nicht gefunden oder nicht eindeutig.
2	Ungültiger Aufruf: unbekannter oder falsch angegebener Schalter, fehlende Regelwerk-Dateien, `--out` mit Dateiendung bei mehreren Tests. Es startet kein Test.

5 Ein- und Ausgabe

Ein Programm wählt einen Kanal mit einem Anwahlbefehl (Operation 030 mit Adresse 4400…4700) aus. Danach wirken Eingabe- und Ausgabebefehle auf das zugehörige Gerät.

Anwahl	Kanal	Eingabe	Ausgabe	Schalter
`4700 030`	K0	Pult-Tastatur (T) oder Schreibwerk (SW)	keine	`--k0`
`4600 030`	K1	Lochstreifenleser 1	Lochstreifenstanzer 1	`--leser1`, `--stanzer1`
`4500 030`	K2	Lochstreifenleser 2	Schreibwerk (Terminal)	`--leser2`, `--sw-ausgabe`, `--SW-TERMINAL`
`4400 030`	K3	frei	frei	– (nicht benutzt)

Ein Ausgabe-Halt-Befehl (`a 030` mit $a_1 \le 3$) gibt das Zeichen aus, dessen Ausgabe-Adresse (die ersten drei Oktalstellen von `a`, Kodetabelle 1) passt. Während der Ausgabe ist Flag A = 1. Der Stanzer wertet alle acht Bits aus, das Schreibwerk nur sechs (Spuren v u 8 4 2 1; die Spuren w und p ändern das Zeichen nicht).

> **Achtung** Ein Ausgabe-Halt ohne angewähltes Ausgabegerät (Kanal K0, K3 oder gar keine Anwahl) liegt außerhalb des definierten Betriebs: Der Rechner würde mit gesetztem Flag A stoppen, der Interpreter meldet das als Stub und lässt A = 1 (Endzeile `A=1`). Dasselbe gilt für eine nicht existierende Anwahl. Kanal K3 hat kein Gerät und wird in dieser Ausbaustufe nie benutzt.

5.1 Schreibwerk-Ausgabe (K2)

- Die Schreibwerk-Ausgabe läuft immer über Kanal K2; die Stellung des Schalters SW hat keinen Einfluss (SW betrifft nur die Eingabeseite, Kanal K0).
- Ohne `--sw-ausgabe` erscheint jedes Zeichen sofort auf dem Terminal in Dunkelgelb.
- Mit `--sw-ausgabe PFAD` erscheint nichts am Terminal; die Zeichen gehen in `PFAD/sw-ausgabe.txt` (bei der ersten Ausgabe eines Tests neu angelegt, danach angehängt). `WRZL` schließt die aktuelle Zeile ab.
- Mit `--SW-TERMINAL` erscheint jedes Zeichen **zusätzlich** im Fenster „SW Ausgabe“ (nur Text, 130 × 36 Zeichen, verschiebbar, Vorgabe rechts unten auf dem Desktop), gleichgültig ob `--sw-ausgabe` gesetzt ist; Datei bzw. Terminal bleiben wie beschrieben. Das Fenster läuft als eigener Prozess und überlebt den Interpreter und sein Terminal.
- Die Zeichenlage KB/GB wird beim Start zufällig gesetzt. `Um. KB` (Ausgabe-Adresse `117x`) und `Um. GB` (`057x`) stellen sie um; sie wirken, geben aber selbst nichts aus. Ohne Umschaltzeichen im Programm ist die Groß-/Kleinschreibung also zufällig.
- Zeichen ohne sichtbare Darstellung erscheinen als `<n>` (z. B. `<Irrung Zeichen>`); `SatzMarke` (`272x`) ist ein Zeilenumbruch.
- Ist eine Ausgabe-Adresse nicht in Tabelle 1, erscheint nichts (Hinweis im Protokoll).
- Bei `STUFE=1` mit `--knotentest` erscheinen die Knotenzeilen erst nach dem Lauf; die Schreibwerk-Ausgabe steht davor.

> **Hinweis Fenster „SW Ausgabe“** (`--SW-TERMINAL`)**.** Das Fenster bleibt **immer offen**, auch nach dem Ende des Tests, bis Sie es mit Klick auf X schließen. Es läuft als eigener Prozess; jeder Aufruf öffnet ein neues Fenster. Text im Fenster lässt sich mit der Maus markieren und kopieren: sofort beim Loslassen der Maustaste, mit Strg+C oder Strg+Einfg oder über das Kontextmenü (rechte Maustaste). Das gilt unter Linux und unter Windows.

Markieren und Kopieren im Fenster

Der Text im Fenster lässt sich mit der Maus markieren (auch Doppel- und Dreifachklick). Kopiert wird sofort beim **Loslassen der Maustaste**, mit **Strg+C**, **Strg+Umschalt+C** oder **Strg+Einfg** (auch wenn der Tastaturfokus nicht im Textfeld liegt) und über das **Kontextmenü** (rechte Maustaste; auf macOS auch mittlere Taste bzw. Strg+Klick) mit „Kopieren“ und „Alles markieren“. Strg+A markiert den gesamten Text. Jedes erfolgreiche Kopieren meldet sich kurz in der Titelleiste („kopiert (N Zeichen, Verfahren)“), ein Fehlschlag mit „Kopieren fehlgeschlagen“ (Kapitel 7). Derselbe Text wird innerhalb von 5 Sekunden nicht erneut in die Zwischenablage gelegt.

Das Verfahren wählt `--zwischenablage`. Mit `auto` (Vorgabe) wird zuerst ein Systemprogramm benutzt, das die Zwischenablage dauerhaft übernimmt: unter Windows die Win32-API, unter Linux `wl-copy` (Wayland), `xsel` oder `xclip` (X11), unter macOS `pbcopy`, unter WSL zusätzlich `clip.exe`. Die Auswahl erfolgt bei jedem Kopieren nach den

Umgebungsvariablen und den installierten Programmen; fehlende werden übersprungen. Die Tk-Zwischenablage ist der Rückfall. In virtuellen Maschinen entscheidet das Gast-Zusatzprogramm, wie der Text zum Host kommt; probieren Sie dort `--zwischenablage system` oder `tk`.

Beispiele (die Fenster selbst lassen sich hier nicht abbilden; im Fenster steht `Jahr 1972`, das Terminal zeigt weiter die Zeilen des Interpreters):

```
$ python3 c8205_interp.py --SW-TERMINAL Tests/0015-030.txt
$ python3 c8205_interp.py --SW-TERMINAL --zwischenablage system Tests/0015-030.txt
$ python3 c8205_interp.py --SW-TERMINAL --sw-ausgabe virtuelle_Geraete/SW-Ausgabe Tests/0015-030.txt
```

5.2 Lochstreifenleser (K1, K2)

`--leser1` und `--leser2` nehmen eine **Datei** (ein Streifen) oder einen **Ordner** (Streifen in Namensreihenfolge). Pro Eingabebefehl wird eine Zeile gelesen: acht Zeichen `0`/`1` für die Spuren `w v u p 8 4 2 1` (die Transportspur wird nicht dargestellt). Leerzeilen sind Vorspann und werden übergangen.

- Ungerade Zahl gelochter Spuren = gültig. Gerade Zahl = Fehler bei Eingabe: Flag FE := 1, der Lauf hält.
- Ausnahme „Irrung Zeichen“ (`11111111`, alle acht Spuren): keine Paritätsprüfung, kein FE, nicht ins Adressregister übernommen. Der Eingabebefehl überspringt die Zeile und liest das nächste Zeichen.
- Die Eingabeadresse wird aus dem Lochbild berechnet und muss in Tabelle 1 vorkommen; sie wird mit dem Adressregister ODER-verknüpft.
- Danach wird der Befehl umgestellt: bei Sprung-Umstellung „ja“ (z. B. `:`, `.`, `,`, Tabulator, `Um. KB`, `Um. GB`) zu einem unbedingten Sprung an `a ODER Eingabeadresse`; bei „nein“ (Ziffern, Buchstaben) zu einem Lesebefehl, der den **Inhalt der Zelle** `a ODER Eingabeadresse` zum AC addiert (nicht die Eingabeadresse selbst).
- Ist der Streifen zu Ende, wartet der Interpreter still (ohne Meldung) auf eine neu in den Pfad kopierte Streifendatei (der reale Leser würde ebenfalls warten); ein nicht vorhandener Pfad verhindert den Teststart. Strg+C beendet das Warten (Rückkehrcode 8).

> **Achtung** Die Ziffer 5 liefert nicht den Wert 5 im AC, sondern den Inhalt der Zelle 0005 (ODER Adressteil des Eingabebefehls). Ist die Zelle nicht belegt, bleibt der AC 0. Legen Sie die gewünschten Konstanten in den Trommelabschnitt, auch wenn der Wert 0 ist: Die Trommel behält Reste früherer Läufe (Abschnitt 3.7). Beispiel in Abschnitt 8.3.

5.3 Lochstreifenstanzer (K1)

`--stanzer1 PFAD` ist die Ausgabedatei (bei einem Ordner: `stanzer-ausgabe.txt` darin). Je gestanztem Zeichen entsteht eine Zeile mit acht Zeichen `0`/`1` für `w v u p 8 4 2 1`. Die Datei wird beim ersten gestanzten Zeichen eines Laufs neu angelegt (nicht angehängt) und nur angelegt, wenn wirklich gestanzt wird. Anders als beim Schreibwerk gibt es keinen Terminal-Ausweg: Ohne `--stanzer1` endet ein Test, der auf K1 ausgibt, mit der roten Fehlermeldung aus Kapitel 7.

5.4 Pult-Tastatur und Schreibwerk-Eingabe (K0)

Ohne `--k0` liest der Interpreter einen Tastendruck je Eingabebefehl vom Terminal (ohne Echo, ohne Enter; Strg+C wirkt). `--k0 datei=PFAD` (oder die gleichnamige Zeile der Testdatei) liest eine Textdatei mit einem Zeichen oder Namen je Zeile (`ZR`, `WM`, `IS`, `IB`, `TAB`, `WRZL` oder Namen aus Tabelle 1; Zeilen mit `#` sind Kommentar). `PFAD` darf ein Ordner mit mehreren Dateien sein, die in alphanumerischer Reihenfolge gelesen werden. Ist die Eingabe zu Ende, wartet der Interpreter still auf eine neu in den Pfad kopierte Datei; ein nicht vorhandener Pfad verhindert den Teststart.

Zustand	Eingebbar
SW nicht gedrückt (Flag T, Pult-Tastatur)	nur `0`–`9`, `+`, `-`, `,`, `.`, `:`, `ZR`, `WM`, `IS`, `IB` (19 Codes). `B` und `D` haben keinen Code und sind nicht eingebbar.
SW gedrückt (Flag SW, Schreibwerk)	alle Zeichen der Tabelle 1

Ein nicht erlaubtes Zeichen wird bei `datei=` mit roter Meldung abgelehnt (der Test endet); bei Terminal-Eingabe wird der Tastendruck ignoriert und auf die nächste gültige Taste gewartet. Zeichen werden über die Spalten „Ascii KB/GB“ der Kodetabelle gefunden; die Eingabe-Zeichenlage (Start KB, geändert durch gelesenes `Um. KB`/`Um. GB`) bestimmt, welche Spalte zuerst durchsucht wird. Nur bei Terminal-Eingabe ohne weitere Zeichen (z. B. geschlossenes stdin) endet der Lauf mit `EINGABE_ENDE`:

```
$ python3 c8205_interp.py Tests/k0-ziffer.txt < /dev/null; echo "Rückkehrcode: $?"
=== Test k0-ziffer (#PROT STUFE=0) ===
Ende: EINGABE_ENDE: K0-Eingabequelle zu Ende (Flag E bleibt 1) | Knoten: 37 | BZ=0002 AC=00000000000 AR=0000 |
    FE=0 FUE=0 BS=0 US=0 E=1 A=0 V1=0 V2=0
Rückkehrcode: 4
```

Echo im Fenster. Mit `--SW-TERMINAL` erscheint jedes **angenommene** K0-Zeichen (T oder SW; vom Terminal oder aus `--k0 datei=PFAD`) zusätzlich als Echo in Dunkelgelb im Fenster „SW Ausgabe“; die Ausgaben des Rechners stehen dort in der Standardfarbe, so lassen sich Eingabe und Ausgabe unterscheiden. Das Echo zeigt das Zeichen wie das Schreibwerk (Namen ohne Zeichen als `<n>`) in der gerade gültigen Zeichenlage. Nicht angenommene Zeichen und die Eingaben der Lochstreifenleser erscheinen nicht. Ohne den Schalter gibt es kein Echo.

> **Hinweis** Unter Linux/macOS wird `termios`, unter Windows `msvcrt.getch()` benutzt. Unter Windows kann Strg+C in Ausnahmefällen als normales Zeichen ankommen; im Zweifel hilft Strg+Pause oder das Schließen des Konsolenfensters.

6 Ergebnisse lesen

6.1 Bildschirmausgabe

Je Test erscheinen der Reihe nach: die Kopfzeile `=== Test <n> (#PROT STUFE=n) ===`, die Schreibwerk-Zeichen des Programms, eine eventuelle Diagnose (`-- Diagnose --`), bei Stufe 1 und `--knotentest` die Knotenzeilen, die Endzeile und – falls der Test Prüfungen enthält – die Ergebnisprüfung.

6.2 Die Endzeile

```
Ende: <Grund> | Knoten: n | BZ=… AC=… AR=… | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Teil	Bedeutung
<Grund>	warum der Lauf endete (Tabelle unten)
Knoten: n	Zahl der durchlaufenen Knoten (fassungsabhängig, kein Sollwert, keine Zeitangabe)
BZ, AR	Befehlszähler und Adressregister, vierstellig oktal
AC	Akkumulator, elfstellig oktal
FE	Fehler bei Eingabe
FUE	Überlauf (FÜ)
BS, US	bedingter bzw. unbedingter Stopp
E, A	Eingabe- bzw. Ausgabebefehl aufgestellt (der Lauf wartet noch auf ein Gerät; bei einem Ausgabe-Stub bleibt A = 1)
V1, V2	Vergleicher (V3 = V1 und V2)

Endgrund	Bedeutung
Unbedingter Stop (US:=1)	Programm hat ordnungsgemäß gehalten
Bedingter Stop (BS:=1)	bedingter Stopp-Befehl bei eingerastetem Schalter BS
Halt wegen Ueberlauf (FUE bereits 1)	Überlauf-Halt; weder US noch BS gesetzt, nur FÜ
Fehler bei Eingabe (FE)	gerade Zahl von Lochungen gelesen
EINGABE_ENDE	Terminal-Eingabe (K0) ohne weitere Zeichen, z. B. geschlossenes stdin. Bei Leser und `--k0 datei=` gibt es kein Ende, sondern stilles Warten.
Halt (Schalter H): Tastenfolge endet ohne R	Tastenfolge endete im Halt (gültig, z. B. `H G Ü E`)
Halt (Schalter H): INTERRUPT nach Befehl pppp endet ohne R/ST	Die Tastenfolge eines `#INTERRUPT` endete weder mit `R` noch mit `ST` (gültig; Abschnitt 3.2)

Endgrund	Bedeutung
Geraet nicht nutzbar ...	Gerät nicht angegeben oder Datei/Zeichen ungültig (rote Meldung davor; davor steht auch die Zeile „TEST NICHT ORDNUNGSGEMAESS BEENDET“). Gilt nicht für K0 (Terminal) und K2-Ausgabe (Terminal).
STUB bei Knoten n ...	Regelwerk unvollständig oder außerhalb des definierten Betriebs: Kanal K3 benutzt, Ausgabe-Halt ohne Gerät (Kapitel 7)
Abbruch: Knotenlimit erreicht (Endlosschleife?)	Der Lauf hat die Sicherheitsgrenze von über 2 Millionen Knoten überschritten: Das Programm hält nie an (Kapitel 7).
Tastenfolge nicht gueltig/definiert	Test startete nicht (Abschnitt 3.3)

Beispiel für den Endgrund „Halt wegen Überlauf“: Der Test addiert zweimal 1 zu 37777777777, der größten positiven Zahl; nach dem Überlauf hält der Rechner, es ist weder US noch BS gesetzt, nur FUE=1.

```
$ python3 c8205_interp.py Tests/ueberlauf.txt; echo "Rückkehrcode: $?"
=== Test ueberlauf (#PROT STUFE=0) ===
Ende: Halt wegen Ueberlauf (FUE bereits 1) | Knoten: 71 | BZ=0003 AC=40000000000 AR=3001 | FE=0 FUE=1 BS=0 US=0
    E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

6.3 Diagnose bei Befehl 164

Der Leerbefehl 164 löst die Diagnose aus, wenn für seine Adresse xxxx Testmarken definiert sind. Reihenfolge: zuerst die #DOK-Texte (Dateireihenfolge), dann alle #SPEI-Bereiche, dann alle #SPUR-Spuren; jede Speicherzelle steht als aaaa: wwwwwwwwww. Vor #SPEI/#SPUR steht die auslösende Marke selbst als Zeile. Die Marken sind unabhängig voneinander (auch ein einzelnes #SPEI ohne #DOK gibt seine Zellen aus); passt keine Marke, entfällt die Diagnose ganz. Ein Beispiel mit #SPEI zeigt Abschnitt 8.4. Eine #SPUR-Marke gibt alle 32 Zellen der Spur in der Reihenfolge aus, in der sie unter dem Kopf der Trommel vorbeilaufen (die Adressen springen: 4000, 4015, 4032, 4007 …, nicht 4000, 4001, 4002 …); im Beispiel steht in der Zelle 4001 der Wert 7:

```
$ python3 c8205_interp.py Tests/spur-demo.txt
=== Test spur-demo (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
```

Achtung Die Leer-Befehle 164 und 166 laufen ohne Resultatbehandlung: Das Z-Feld erzeugt keine Verschiebung des AC.

6.4 Knotenprotokoll und Protokolldatei

Bei #PROT STUFE=1 erscheinen die Knotenzeilen auf dem Bildschirm, bei STUFE=2 in der Protokolldatei – aber nur, wenn Sie mit --knotentest aufrufen. Ohne den Schalter entfallen die Zeilen Kxxx …; erhalten bleiben die BR=-Zeilen (vor Knoten 2 und 7), die Diagnose sowie Fehler- und Stub-Meldungen.

```
$ python3 c8205_interp.py --knotentest Tests/kurz.txt
=== Test kurz (#PROT STUFE=1) ===
-- Knotenzustaende --
K001 KON B BZ=0000 AR=0000 AC=00000000000
  BR=0000 000  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0
K002 PHA Phase I BZ=0000 AR=0000 AC=00000000000
K003 ANW 001 BZ=0001 AR=0000 AC=00000000000
K004 TST 002 BZ=0001 AR=0000 AC=00000000000 -> falsch
K005 TST 003 BZ=0001 AR=0000 AC=00000000000 -> wahr
K008 PHA Phase II BZ=0001 AR=0000 AC=00000000000
K009 ANW 005 BZ=0001 AR=0000 AC=00000000000
K010 TST 006 BZ=0001 AR=0000 AC=00000000000 -> falsch
K011 ANW 007 BZ=3000 AR=0000 AC=00000000000
K012 TST 008 BZ=3000 AR=0000 AC=00000000000 -> falsch
K015 PHA Phase III BZ=3000 AR=0000 AC=00000000000
K016 ANW 001 BZ=0001 AR=3000 AC=00000000000
…
```

Auszug: die ersten 15 Zeilen der Ausgabe.

Zeile	Lesart
`Kxxx ANW nnn BZ=… AR=… AC=…`	Knoten xxx, Typ Anweisung (ANW), ANWEISUNG-nnn; danach BZ, AR und AC, jeweils oktal
`Kxxx TST nnn … -> falsch`	Knoten xxx, Typ Test (TST), ANWEISUNG-nnn; `-> wahr` oder `-> falsch` ist das Testergebnis
`KON, PHA`	Konnektor bzw. Phasenzuordnung („Phase I“ … „Phase IV“)
`BR=aaaa ooo (vor Knoten 002) …`	Befehlsregister (Adressteil und Operationsteil des geladenen Befehls) vor Beginn von Phase I; dazu BZ, AC, Überlauf FÜ und die Vergleicher V1, V2, V3

Die Protokolldatei (nur bei `STUFE=2`) hat bis zu vier Abschnitte: `== 1. Ausgangsbedingungen ==` (die vom Test geschriebenen Zellen, Anfangswerte, Tastenfolge mit der Wirkung jeder Taste), `== 2. Zustaende des Knotenautomaten ==`, `== Ende ==` und, falls der Test Prüfungen enthält, `== 3. Ergebnispruefung ==`. Beispiel einer Tastenfolge mit vielen Token (aus Abschnitt 8.2):

```
$ python3 c8205_interp.py --out Test-Protokolle/tf-demo.log Tests/tf-demo.txt
=== Test tf-demo (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 28 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Protokolldatei: Test-Protokolle/tf-demo.log
$ cat Test-Protokolle/tf-demo.log
Test tf-demo

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00000000020
  #PROT STUFE=2
Tastenfolge: SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 SP=0 R
  SW gedrueckt: Schalter SW -> Flag SW := 1, T := 0
  SW geloest (Stellung T): Flag SW := 0, T := 1
  BS : Schalter BS gedrueckt (eingerastet) -> Flag BS := 1
  SP : Schalter SP gedrueckt (eingerastet) -> Flag SP := 1
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000); US :=
    1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; BZ bleibt unveraendert (BZ := 0 erst bei Ausfuehrung des
    Sprungs nach 0000)
  Ue : AR := 7777; OR := OR ODER 240 -> BR = 7777 240 (Transport, aufgestellt, nicht ausgefuehrt)
  ST : (US oder BS) OR := OR ODER 240 -> BR = 7777 240, AR unveraendert; US := 0 (aufgestellt, nicht ausgefuehrt)
  BS : Schalter BS geloest -> Flag BS := 0
  SP : Schalter SP geloest -> Flag SP := 0
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B bzw. bei Knoten 19 (nach
    M mit Sprung mit Rueckkehrabsicht bei Knoten 54), wenn BR aufgestellt ist
  Nicht spezifizierte Register (AC, ...) starten mit 0.

== 2. Zustaende des Knotenautomaten ==
  BR=7777 240  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0

== Ende ==
Unbedingter Stop (US:=1)
BZ=0001 AC=00000000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Hinweis Die Kopfzeile des Abschnitts 1 lautet „Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0)“. Das ist ein Rest aus der Zeit vor der persistenten Trommel: Nicht genannte Zellen haben nicht unbedingt den Wert 0, sondern den Inhalt der Trommeldatei (Abschnitt 3.7). Das Protokoll zeigt nur die Zellen des Trommelabschnitts.

Zeilen für `#AC=` und `#BZ=` erscheinen, wenn die Marken gesetzt sind:

```
$ python3 c8205_interp.py --out Test-Protokolle/ac-log.log Tests/ac-log.txt
=== Test ac-log (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0012 AC=00000012000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Protokolldatei: Test-Protokolle/ac-log.log
$ cat Test-Protokolle/ac-log.log
Test ac-log

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0011: 00000000020
  #AC=00000000100
  #BZ=0010
  #PROT STUFE=2
Tastenfolge: H M-drücken E M-loslassen R
  H  : Schalter H-R -> "H", Flag H := 1
  M gedrueckt halten: OR := OR ODER 025 (BR = AR 025)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0011, AR := 0011, AC := 00000012000 (BZ+1 im Adressfeld)
  M loslassen: OR-Verknuepfung mit 025 entfaellt fuer Folgeschritte
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B
  Anfangswerte: AC = 00000000100 (#AC), BZ = 0010 (#BZ)
  Nicht spezifizierte Register (AR, OR, ...) starten mit 0.

== 2. Zustaende des Knotenautomaten ==
  BR=0000 000  (vor Knoten 002)  BZ=0011 AC=00000012000 FÜ=0 V1=0 V2=0 V3=0

== Ende ==
Unbedingter Stop (US:=1)
BZ=0012 AC=00000012000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

6.5 Ergebnisprüfung

Hat die Testdatei einen `Prüfungen:`-Teil und endete der Lauf ordnungsgemäß, folgt auf die Endzeile die Prüfung: je Wert Soll und Ist mit OK oder FEHLER, danach `Test wahr` oder `Test falsch`. Sie erscheint unabhängig von `#PROT STUFE`; bei `STUFE=2` steht dasselbe als Abschnitt `== 3. Ergebnispruefung ==` am Ende der Protokolldatei.

```
$ python3 c8205_interp.py Tests/pruef-falsch.txt; echo "Rückkehrcode: $?"
=== Test pruef-falsch (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0003 ist=0004 FEHLER
  AC: soll=00000000145 ist=00000000145 OK
Test falsch
Rückkehrcode: 6
```

`Test wahr` gibt es nur, wenn alle angegebenen Prüfungen übereinstimmen. Hier stimmt BZ nicht (soll 0003, ist 0004); der AC stimmt. Rückkehrcode 6 (Abschnitt 4.4). Beispiel mit Erfolg: Abschnitt 8.4.

Dateiprüfungen

`--pruefe-stanzer1` und `--pruefe-sw-ausgabe` erscheinen im selben Block hinter den Einzelprüfungen, je Prüfung eine Zeile:

Zeile	Bedeutung
Stanzer 1: soll=<Soll-Datei> ist=<Ist-Datei> OK	Inhalt (zeilenweise) gleich; für das Schreibwerk steht „Schreibwerk:“ davor.
… FEHLER (Soll-Datei fehlt)	Die Soll-Datei existiert nicht (Name ohne Ordner: neben der Testdatei suchen).
… FEHLER (Ist-Datei fehlt, vom Lauf nicht angelegt)	Der Lauf hat nichts gestanzt bzw. ausgegeben; eine übrig gebliebene Datei eines früheren Laufs zählt nicht.
… FEHLER (Inhalt verschieden, diff):	Darunter der Unterschied im Format `diff -u` (Soll gegen Ist, höchstens 100 Zeilen, danach „… n weitere diff-Zeilen nicht angezeigt“).

Beispiel: Der Stanzer-Test aus Abschnitt 8.3 gegen eine Soll-Datei, die in der letzten Zeile abweicht:

```
$ python3 c8205_interp.py --pruefe-stanzer1 soll-falsch.txt Tests/stanzer-pruefe.txt; echo "Rückkehrcode: $?"
=== Test stanzer-pruefe (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  Stanzer 1: soll=Tests/soll-falsch.txt ist=out/band.txt FEHLER (Inhalt verschieden, diff):
    --- soll Tests/soll-falsch.txt
    +++ ist out/band.txt
    @@ -1,2 +1,2 @@
     00010101
    -00111111
    +00111011
Test falsch
Rückkehrcode: 6
```

Der Rückkehrcode folgt dem Gesamtergebnis (0 oder 6). Mit `--NO-TERMOUT` entfällt die Bildschirmausgabe; das Ergebnis steht dann bei `#PROT STUFE=2` in der Protokolldatei und im Rückkehrcode.

7 Wenn etwas schiefgeht

Gerätefehler erscheinen in Rot auf dem Terminal (Standardfehlerausgabe, nur mit Terminal gefärbt) und beginnen mit „FEHLER:“. Danach folgt die Zeile „TEST NICHT ORDNUNGSGEMAESS BEENDET: …“ und die Endzeile. Meldungen der Vorab-Prüfungen (Testdatei, Trommeldatei, Aufruf) erscheinen ohne Farbe. Das Fehlerverzeichnis ist alphabetisch nach dem Meldungstext geordnet; die letzte Spalte nennt den Rückkehrcode (Abschnitt 4.4).

Meldung (Anfang)	Ursache	Abhilfe	Code
Abbruch: Knotenlimit erreicht (Endlosschleife?)	Programm läuft ohne Stopp-Befehl weiter, z. B. weil eine leere Zelle (Wert 0 = Sprung nach 0000) erreicht wird	Stopp-Befehl `00000000020` hinter das Programmende schreiben; Programmfluss prüfen	8
Ausgabe auf Lochbandstanzer 1 (Kanal K1) angefordert, aber --stanzer1 nicht angegeben.	Programm stanzt, Schalter fehlt	`--stanzer1 PFAD` angeben (Schalter oder Zeile)	8
Das Fenster „SW Ausgabe“ bleibt nach dem Test offen	gewollt: Das Fenster schließt sich nie von selbst	mit Klick auf X schließen (jeder Aufruf mit `--SW-TERMINAL` öffnet ein eigenes Fenster)	–
Eingabe von Lochbandleser n (Kanal Kn) angefordert, aber --lesern nicht angegeben.	Programm liest von K1/K2, Schalter fehlt	`--leser1` bzw. `--leser2` angeben (Schalter oder Zeile)	8
FEHLER: Trommeldatei "…" ist formal fehlerhaft - Test(s) starten nicht: mit Zeilen wie „Trommeldatei nicht gefunden: …“, „Trommeldatei hat n Zeilen; erwartet werden genau 4096 …“, „Zeile n: ungueltiges Format …“, „Zeile n: Adresse aaaa erwartet, gefunden bbbb.“, „Trommeldatei hat CRLF-Zeilenenden …“	Trommeldatei fehlt, hat nicht genau 4096 Zeilen, eine Zeile hat das falsche Format, die Adressen sind nicht lückenlos aufsteigend oder die Zeilenenden sind CRLF statt LF	`--trommel` bzw. Pfad prüfen; leere Trommel mit `python3 erzeuge_leere_trommel.py` neu erzeugen (Abschnitt 2.3); es startet kein Test	16
FEHLER: Trommeldatei "…" kann nicht geoeffnet werden: …	Datei lässt sich nicht zum Schreiben öffnen (Rechte, schreibgeschützt)	Schreibrechte prüfen; es startet kein Test	16
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:	Testdatei formal fehlerhaft (Zeilenformat, doppelte Adresse, `#SPEI`-Spanne, `#PROT`, `#AC`/`#BZ`, `#INTERRUPT`, `#STARTE`, Tastenfolge, `Prüfungen:`, Geräte-Zeile oder -Pfad)	die genannte Zeile korrigieren (Abschnitt 3.5)	16
K0-Datei nicht lesbar: …	Datei fehlt oder ist nicht lesbar	Pfad prüfen	8

Meldung (Anfang)	Ursache	Abhilfe	Code
K0-Eingabe: '...' ist an der Pult-Tastatur (T) nicht eingebbar ...	Zeichen ist an T nicht erlaubt (z. B. B, Buchstaben)	`SW=1` vor `G` in die Tastenfolge oder ein anderes Zeichen	8
--k0: 'datei=PFAD' erwartet ...	ungültiger Wert für `--k0` (Schalter: Code 2; Zeile der Testdatei: Code 16)	`--k0 datei=PFAD` schreiben	2 / 16
K0: Eingabe-Pfad nicht gefunden: ...	bei `--k0 datei=PFAD` (Schalter oder Zeile) existiert der Pfad nicht	Pfad prüfen	16
Knotenbeschreibung nicht gefunden: ... / Knotengraph nicht gefunden: ...	eine der beiden Regelwerk-Dateien fehlt im `--mdverz`-Verzeichnis	`--mdverz` korrekt angeben oder Datei dorthin legen	2
Kopieren fehlgeschlagen (Titelleiste des Fensters „SW Ausgabe“)	Kein Kopierverfahren hat funktioniert (Linux: weder `wl-copy`/`xsel`/`xclip` noch Tk-Zwischenablage erreichbar; Windows: Zwischenablage von einem anderen Programm gesperrt)	`--zwischenablage system` bzw. `tk` versuchen; unter Linux `xsel`, `xclip` oder `wl-clipboard` installieren; in virtuellen Maschinen das Gast-Zusatzprogramm prüfen	–
Lochbandleser n: Lochstreifen-Pfad nicht gefunden: ...	Pfad falsch	Pfad prüfen	16
Lochstreifen ...: ungueltige Zeile ...	Zeile ist nicht acht Zeichen aus `0`/`1`	Streifendatei korrigieren	8
--out mit Dateiendung ... ist nur mit genau einer Testdatei zulaessig	`--out x.log` zusammen mit mehreren Tests	Ordner ohne Endung verwenden oder Tests einzeln starten	2
--pruefe-stanzer1 ohne --stanzer1: ... / --pruefe-sw-ausgabe ohne --sw-ausgabe: ...	Dateiprüfung ohne das zugehörige Gerät; es gäbe keine Ist-Datei	passenden Geräteschalter ergänzen (Schalter oder Zeile der Testdatei)	16
--pruefe-stanzer1/--pruefe-sw-ausgabe sind nur mit genau einer Testdatei zulaessig	Dateiprüfung mit mehreren Testdateien	Tests einzeln starten	2
STUB bei Knoten n: ...	eine vom Programm benutzte Funktion ist nicht umgesetzt oder liegt außerhalb des definierten Betriebs (siehe unten)	Meldung und Testdatei notieren; Programm prüfen	8
Tastenfolge nicht gueltig/definiert	unbekanntes Token, weder `R` noch Halt am Ende, M nicht losgelassen	Abschnitt 3.3	16
Testdatei nicht gefunden: ...	Name weder als Pfad noch unterhalb der Suchordner vorhanden	Pfad, Schreibweise und aktuellen Ordner prüfen (wird mit angezeigt)	16
Testdatei ... nicht eindeutig gefunden	mehrere Dateien gleichen Namens	vollständigen Pfad angeben	16
WARNUNG: --SW-TERMINAL: kein Ausgabefenster moeglich: ...	`tkinter` fehlt oder es gibt keine grafische Oberfläche (unter Linux `DISPLAY` nicht gesetzt)	`python3-tk` installieren bzw. in einer grafischen Sitzung starten; der Test läuft trotzdem, nur ohne Fenster	Code des Tests
Zeile n: #INTERRUPT: ...	Formfehler einer `#INTERRUPT`-Zeile (z. B. vor der ersten Befehlszeile, erste Taste nicht `H`)	Zeile korrigieren (Abschnitt 3.2)	16
Zeile n: --leser1/--leser2/--stanzer1/--sw-ausgabe/--k0 ...	Formfehler einer Geräte-Zeile der Testdatei: ohne Wert, mehrfach, `--k0` ohne `datei=`	Zeile korrigieren; jede Zeile höchstens einmal (Abschnitt 3.6)	16
Zeile n: --SW-TERMINAL ist nur als Kommandozeilenschalter zulaessig, nicht in der Testdatei	Der Schalter steht als Zeile in der Testdatei	Zeile entfernen; `--SW-TERMINAL` beim Aufruf angeben	16

Stub. Ein Stub ist kein Fehler des Aufrufs, sondern die Meldung „diese Stelle des Regelwerks ist nicht umgesetzt“ bzw. „außerhalb des definierten Betriebs“. Für die 512 Operationscodes gibt es keinen Stub mehr. Ein Stub tritt auf, wenn ein Programm auf Kanal K3 ein- oder ausgibt, wenn ein Ausgabe-Halt ohne Ausgabegerät ausgeführt wird (Kanal K0, K3 oder keine Anwahl; A bleibt 1), wenn ein Anwahlbefehl mit $a_2 < 4$ benutzt wird oder wenn eine Eingabeadresse in Tabelle 1 nicht vorkommt. Notieren Sie Meldung und Testdatei; der Test wurde nicht „repariert“.

Beispiele

Gerät nicht angegeben. Der Test aus Abschnitt 8.3 liest von Leser 1; ohne `--leser1` endet er mit der roten Meldung, Rückkehrcode 8:

```
$ python3 c8205_interp.py Tests/leser-ziffer.txt; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
FEHLER: Eingabe von Lochbandleser 1 (Kanal K1) angefordert, aber --leser1 nicht angegeben.
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 48): --leser1 nicht angegeben
Ende: Geraet nicht nutzbar (Knoten 48): --leser1 nicht angegeben | Knoten: 37 | BZ=0002 AC=00000000000 AR=0000 |
    FE=0 FUE=0 BS=0 US=0 E=1 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Entsprechend für den Stanzer (Test `stanzer-zeichen.txt`, Abschnitt 8.3) ohne `--stanzer1`; der Ausgabebefehl steht aufgestellt (A=1):

```
$ python3 c8205_interp.py Tests/stanzer-zeichen.txt; echo "Rückkehrcode: $?"
=== Test stanzer-zeichen (#PROT STUFE=0) ===
FEHLER: Ausgabe auf Lochbandstanzer 1 (Kanal K1) angefordert, aber --stanzer1 nicht angegeben.
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 23): --stanzer1 nicht angegeben
Ende: Geraet nicht nutzbar (Knoten 23): --stanzer1 nicht angegeben | Knoten: 38 | BZ=0002 AC=00000000000 AR=0250 |
    FE=0 FUE=0 BS=0 US=0 E=0 A=1 V1=0 V2=0
Rückkehrcode: 8
```

Zeichen an der Pult-Tastatur nicht eingebbar. Die Datei `band/k0b.txt` enthält das Zeichen `B`; der Test benutzt die Pult-Tastatur (T), an der `B` keinen Code hat:

```
$ python3 c8205_interp.py --k0 datei=band/k0b.txt Tests/k0-ziffer.txt; echo "Rückkehrcode: $?"
=== Test k0-ziffer (#PROT STUFE=0) ===
FEHLER: K0-Eingabe: 'B' ist an der Pult-Tastatur (T) nicht eingebbar - nur 0-9 + - , . : ZR WM IS IB (fuer alle
    Codes: Schalter SW druecken).
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 48): 'B' ist an der Pult-Tastatur (T) nicht
    eingebbar - nur 0-9 + - , . : ZR WM IS IB (fuer alle Codes: Schalter SW druecken)
Ende: Geraet nicht nutzbar (Knoten 48): 'B' ist an der Pult-Tastatur (T) nicht eingebbar - nur 0-9 + - , . : ZR WM
    IS IB (fuer alle Codes: Schalter SW druecken) | Knoten: 37 | BZ=0002 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0
    US=0 E=1 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Stub: Ausgabe ohne angewähltes Gerät. Das Programm gibt die Ziffer 5 aus, ohne vorher einen Kanal anzuwählen. Nach dem Original hält der Rechner mit Flag A; der Interpreter meldet das als Stub und lässt A = 1:

```
$ python3 c8205_interp.py Tests/stub-ausgabe.txt; echo "Rückkehrcode: $?"
=== Test stub-ausgabe (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: STUB bei Knoten 23: Ausgabe-Halt ohne Ausgabegeraet (Kanal K0, K3 oder keine
    Anwahl); nach der Originaldokumentation 4.3.2.5 stoppt der Rechner mit gesetztem Flag A (Lampe A)
Ende: STUB bei Knoten 23: Ausgabe-Halt ohne Ausgabegeraet (Kanal K0, K3 oder keine Anwahl); nach der
    Originaldokumentation 4.3.2.5 stoppt der Rechner mit gesetztem Flag A (Lampe A) | Knoten: 19 | BZ=0001
    AC=00000000000 AR=0250 | FE=0 FUE=0 BS=0 US=0 E=0 A=1 V1=0 V2=0
Rückkehrcode: 8
```

Tastenfolge ungültig. Hier fehlt vor `M-` ein `M+` (Test `tf-m.txt`, Tastenfolge `H G M- R`):

```
$ python3 c8205_interp.py Tests/tf-m.txt; echo "Rückkehrcode: $?"
=== Test tf-m (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: Tastenfolge nicht gueltig/definiert
Ende: Tastenfolge nicht gueltig/definiert | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=0 E=0
    A=0 V1=0 V2=0
Rückkehrcode: 16
```

Programm ohne Stopp (Endlosschleife). Das Programm lädt eine Zahl und läuft dann in die leere Zelle 0001 (Wert 0). Ein Befehl mit dem Wert 0 ist der unbedingte Sprung nach Adresse 0000: Das Programm beginnt von vorn und endet nie. Nach über zwei Millionen Knoten bricht der Interpreter ab.

```
$ python3 c8205_interp.py Tests/endlos.txt; echo "Rückkehrcode: $?"
=== Test endlos (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: Abbruch: Knotenlimit erreicht (Endlosschleife?)
Ende: Abbruch: Knotenlimit erreicht (Endlosschleife?) | Knoten: 2000001 | BZ=0001 AC=00000000000 AR=3000 | FE=0
    FUE=0 BS=0 US=0 E=0 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Geräte-Zeile ohne Wert. Die Testdatei enthält in Zeile 3 die Zeile `--stanzer1` ohne Pfad:

```
$ python3 c8205_interp.py Tests/geraet-leer.txt; echo "Rückkehrcode: $?"
=== Test geraet-leer (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: --stanzer1 ohne Wert
Rückkehrcode: 16
```

Schalter als Zeile der Testdatei. `--SW-TERMINAL` gilt nur auf der Kommandozeile:

```
$ python3 c8205_interp.py Tests/sw-term-zeile.txt; echo "Rückkehrcode: $?"
=== Test sw-term-zeile (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: --SW-TERMINAL ist nur als Kommandozeilenschalter zulaessig, nicht in der Testdatei
Rückkehrcode: 16
```

Unterbrechung an falscher Stelle. Die Zeile `#INTERRUPT` steht vor der ersten Befehlszeile (Test `int-fehler.txt`):

```
$ python3 c8205_interp.py Tests/int-fehler.txt; echo "Rückkehrcode: $?"
=== Test int-fehler (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: #INTERRUPT: steht vor der ersten Befehlszeile (muss ZWISCHEN zwei Befehlszeilen stehen)
Rückkehrcode: 16
```

Testdatei nicht gefunden.

```
$ python3 c8205_interp.py Tests/gibtsnicht.txt; echo "Rückkehrcode: $?"
FEHLER: Testdatei nicht gefunden: 'Tests/gibtsnicht.txt' (aktuelles Verzeichnis: .)
Rückkehrcode: 16
```

Pfad der K0-Eingabe nicht vorhanden. Ein nicht vorhandener `--leser1`-, `--leser2`- oder `--k0`-Pfad wird schon vor dem Start gemeldet, auch wenn er als Zeile der Testdatei steht:

```
$ python3 c8205_interp.py --k0 datei=band/gibtsnicht.txt Tests/a.txt; echo "Rückkehrcode: $?"
=== Test a (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - K0: Eingabe-Pfad nicht gefunden: band/gibtsnicht.txt
Rückkehrcode: 16
```

Kein Ausgabefenster möglich. Ohne grafische Oberfläche (hier: `DISPLAY` nicht gesetzt) oder ohne `tkinter` läuft der Test trotz `--SW-TERMINAL` weiter, nur ohne Fenster:

```
$ python3 c8205_interp.py --SW-TERMINAL Tests/a.txt; echo "Rückkehrcode: $?"
=== Test a (#PROT STUFE=0) ===
WARNUNG: --SW-TERMINAL: kein Ausgabefenster moeglich: keine grafische Oberflaeche (DISPLAY nicht gesetzt). Der
    Test laeuft ohne Fenster.
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Vorab-Meldungen bleiben sichtbar. Mit `--NO-TERMOUT` entfällt die Kopfzeile `=== Test … ===`, die Meldung der formalen Prüfung bleibt:

```
$ python3 c8205_interp.py --NO-TERMOUT Tests/formal.txt; echo "Rückkehrcode: $?"
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: ungueltiges Zeilenformat: '0000: 000000002'
  - Zeile 5: Adresse 0001 widerspruechlich belegt
Rückkehrcode: 16
```

Regelwerk-Dateien fehlen. Ein Verzeichnis ohne die beiden `.md`-Dateien ergibt Rückkehrcode 2; mit dem richtigen `--mdverz` läuft derselbe Test:

```
$ mkdir -p leer
$ python3 c8205_interp.py --mdverz leer Tests/a.txt; echo "Rückkehrcode: $?"
usage: c8205_interp.py [-h] [--version] [--mdverz VERZ] [--trommel PFAD]
                       [--leser1 PFAD] [--leser2 PFAD] [--stanzer1 PFAD]
                       [--sw-ausgabe PFAD] [--pruefe-stanzer1 DATEI]
                       [--pruefe-sw-ausgabe DATEI] [--k0 datei=PFAD]
                       [--out OUT] [--knotentest] [--NO-TERMOUT]
                       [--SW-TERMINAL] [--zwischenablage auto|tk|system]
                       tests [tests ...]
c8205_interp.py: error: Knotengraph nicht gefunden: leer/C8205-Interpreter-Eingänge_der_Knoten.md
Rückkehrcode: 2
$ python3 c8205_interp.py --mdverz ./markdown-dateien Tests/a.txt; echo "Rückkehrcode: $?"
=== Test a (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Trommeldatei fehlt. Der Interpreter legt sie nie selbst an; er meldet es vor dem ersten Test, auch mit `--NO-TERMOUT`:

```
$ rm -f virtuelle_Geraete/Trommel/trommel.txt
$ python3 c8205_interp.py Tests/a.txt; echo "Rückkehrcode: $?"
FEHLER: Trommeldatei "virtuelle_Geraete/Trommel/trommel.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Trommeldatei nicht gefunden: virtuelle_Geraete/Trommel/trommel.txt
Rückkehrcode: 16
$ python3 c8205_interp.py --NO-TERMOUT Tests/a.txt; echo "Rückkehrcode: $?"
FEHLER: Trommeldatei "virtuelle_Geraete/Trommel/trommel.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Trommeldatei nicht gefunden: virtuelle_Geraete/Trommel/trommel.txt
Rückkehrcode: 16
```

Trommeldatei fehlerhaft. Hier ist die Datei auf 100 Zeilen gekürzt:

```
$ python3 erzeuge_leere_trommel.py
$ mkdir -p trommeln
$ head -n 100 virtuelle_Geraete/Trommel/trommel.txt > trommeln/kurz.txt
$ python3 c8205_interp.py --trommel trommeln/kurz.txt Tests/a.txt; echo "Rückkehrcode: $?"
FEHLER: Trommeldatei "trommeln/kurz.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Trommeldatei hat 100 Zeilen; erwartet werden genau 4096 (Adressen 0000 bis 7777).
Rückkehrcode: 16
```

Trommeldatei mit CRLF-Zeilenenden. Die Datei wurde nachträglich mit Windows-Zeilenenden versehen:

```
$ python3 erzeuge_leere_trommel.py --aus trommeln/crlf.txt
$ sed -i 's/$/\r/' trommeln/crlf.txt
$ python3 c8205_interp.py --trommel trommeln/crlf.txt Tests/a.txt; echo "Rückkehrcode: $?"
FEHLER: Trommeldatei "trommeln/crlf.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Trommeldatei hat CRLF-Zeilenenden (19 Byte je Zeile): trommeln/crlf.txt. Erwartet wird LF (18 Byte je Zeile),
    da der Interpreter Zellen ueber Adresse x 18 positioniert. Datei auf LF umstellen (z. B. mit
    erzeuge_leere_trommel.py neu erzeugen oder dos2unix).
Rückkehrcode: 16
```

8 Beispiele Schritt für Schritt

Alle Beispiele sind lauffähig und mit der auf Seite 2 genannten Version ausgeführt, jeweils nach `python3 erzeuge_leere_trommel.py`. Legen Sie die Dateien wie angegeben an; die Ordner `Tests/` und `band/` im Projektordner sind frei wählbar. Die Zahl hinter „Knoten:“ ist fassungsabhängig.

8.1 Ausgabe auf dem Schreibwerk

Das Grundbeispiel steht in Abschnitt 2.4: Testdatei `Tests/0015-030.txt`, Aufruf mit dem Wrapper, Ausgabe „Jahr 1972“ in Dunkelgelb. Verwendet werden die Token `H`, `G`, `M-drücken`, `E`, `M-loslassen`, `R` und die Marken `#DOK`, `#PROT STUFE=2`. Mit `--sw-ausgabe` erscheint der Text nicht am Terminal, sondern in `SW-Ausgabe/sw-ausgabe.txt`; mit `--NO-TERMOUT` zusätzlich keine Zeile des Interpreters:

```
$ python3 c8205_interp.py --NO-TERMOUT --sw-ausgabe SW-Ausgabe Tests/0015-030.txt
$ cat SW-Ausgabe/sw-ausgabe.txt

Jahr 1972
```

Die Testdatei kann das Gerät selbst mitbringen und die Ausgabedatei gleich prüfen. `Tests/sw-zeile.txt` (Abschnitt 3.6) stellt `Um. KB` voran, damit das Ergebnis nicht von der zufälligen Zeichenlage abhängt; danach folgen die Ziffer 5 und der Punkt. Mit der Zeile `--pruefe-sw-ausgabe sw-soll.txt` im Teil `Prüfungen:` (Soll-Datei neben der Testdatei, Inhalt `5.`) bewertet der Test die Ausgabe selbst; dasselbe leistet der Schalter beim Aufruf:

```
Trommeldatei:
--sw-ausgabe SW-Ausgabe # Schreibwerk-Ausgabe in Datei
#PROT STUFE=0
0000: 00004500030
0001: 00001170030
0002: 00000250030
0003: 00000730030
0004: 00000000020
Tastenfolge:
H G R
Prüfungen:
US=1
--pruefe-sw-ausgabe sw-soll.txt
```

Datei `Tests/sw-pruefe.txt`

```
5.
```

Datei `Tests/sw-soll.txt`

```
$ python3 c8205_interp.py Tests/sw-pruefe.txt; echo "Rückkehrcode: $?"
=== Test sw-pruefe (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 93 | BZ=0005 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  Schreibwerk: soll=Tests/sw-soll.txt ist=SW-Ausgabe/sw-ausgabe.txt OK
Test wahr
Rückkehrcode: 0
$ python3 c8205_interp.py --pruefe-sw-ausgabe sw-soll.txt Tests/sw-zeile.txt; echo "Rückkehrcode: $?"
=== Test sw-zeile (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 93 | BZ=0005 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  Schreibwerk: soll=Tests/sw-soll.txt ist=SW-Ausgabe/sw-ausgabe.txt OK
Test wahr
Rückkehrcode: 0
```

Mit dem Ausgabefenster (`--SW-TERMINAL`, wahlweise mit `--zwischenablage`) erscheint dieselbe Ausgabe zusätzlich in einem eigenen Fenster:

```
$ python3 c8205_interp.py --SW-TERMINAL Tests/0015-030.txt
$ python3 c8205_interp.py --SW-TERMINAL --zwischenablage system Tests/0015-030.txt
```

Im Fenster „SW Ausgabe“ steht `Jahr 1972`. Das Terminal zeigt wie zuvor die Zeilen des Interpreters (ohne `--sw-ausgabe` auch die dunkelgelbe Ausgabe). Mit `--sw-ausgabe virtuelle_Geraete/SW-Ausgabe` geht dieselbe Ausgabe zusätzlich in die Datei `sw-ausgabe.txt`.

> **Hinweis Fenster „SW Ausgabe“** (`--SW-TERMINAL`)**.** Das Fenster bleibt **immer offen**, auch nach dem Ende des Tests, bis Sie es mit Klick auf X schließen. Es läuft als eigener Prozess; jeder Aufruf öffnet ein neues Fenster. Text im Fenster lässt sich mit der Maus markieren und kopieren: sofort beim Loslassen der Maustaste, mit Strg+C oder Strg+Einfg oder über das Kontextmenü (rechte Maustaste). Das gilt unter Linux und unter Windows.

8.2 Alle Bedientasten in einer Tastenfolge

Die Tastenfolge `SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 SP=0 R` benutzt die Token `SW`, `BS`, `SP`, `H`, `G`, `Ü`, `ST` und `R`. Das Programm besteht nur aus einem Stopp-Befehl; wichtig ist das Protokoll (Stufe 2), das die Wirkung jedes Tokens zeigt (Abschnitt 6.4).

```
Trommeldatei:
#PROT STUFE=2
0000: 00000000020
Tastenfolge:
SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 SP=0 R
```

Datei `Tests/tf-demo.txt`

```
$ python3 c8205_interp.py --out Test-Protokolle/tf-demo.log Tests/tf-demo.txt
=== Test tf-demo (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 28 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Protokolldatei: Test-Protokolle/tf-demo.log
```

Das Protokoll zeigt: `ST` wirkt hier, weil `G` zuvor US gesetzt hat (Verriegelung). Durch `Ü` und `ST` steht BR=7777 240 aufgestellt; `R` startet deshalb bei Knoten 19, führt den Transport aus und hält danach.

Kanaltasten K0 bis K3

Die Kanaltasten stehen nach `G`. Die Tastenfolge `H G M K0 K1 K2 K3` endet im Halt (kein `R`); das Protokoll zeigt, was jede Taste einstellt. Ein im Programm ausgeführter Anwahlbefehl (`4400…4700 030`) übersteuert die Einstellung später.

```
Trommeldatei:
#PROT STUFE=2
0000: 00000000020
Tastenfolge:
H G M K0 K1 K2 K3
```

Datei `Tests/kanal-demo.txt`

```
$ python3 c8205_interp.py --out Test-Protokolle/kanal-demo.log Tests/kanal-demo.txt
=== Test kanal-demo (#PROT STUFE=2) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0
    US=1 E=0 A=0 V1=0 V2=0
Protokolldatei: Test-Protokolle/kanal-demo.log
$ cat Test-Protokolle/kanal-demo.log
Test kanal-demo

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00000000020
  #PROT STUFE=2
Tastenfolge: H G M K0 K1 K2 K3
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000); US :=
    1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; BZ bleibt unveraendert (BZ := 0 erst bei Ausfuehrung des
    Sprungs nach 0000)
  M  : (= M gedrueckt, sofort losgelassen) OR := OR ODER 025 -> BR = AR 025
  K0 : Kanaltaste (rastend): Kanal K0 angewaehlt (SW nicht gedrueckt -> T := 1); ein im Programm ausgefuehrter
    Anwahlbefehl uebersteuert diese Einstellung
  K1 : Kanaltaste (rastend): Kanal K1 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert diese
    Einstellung
  K2 : Kanaltaste (rastend): Kanal K2 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert diese
    Einstellung
  K3 : Kanaltaste (rastend): Kanal K3 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert diese
    Einstellung
  Ende der Tastenfolge ohne R: Rechner bleibt im Halt (H)
  Nicht spezifizierte Register (AC, ...) starten mit 0.

== 2. Zustaende des Knotenautomaten ==


== Ende ==
Halt (Schalter H): Tastenfolge endet ohne R
BZ=0000 AC=00000000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

8.3 Zeichen vom Lochstreifen lesen und auf den Stanzer ausgeben

Leser: die Ziffer 5

Programm: `0000: 00004600030` (K1 anwählen), `0001: 00000000011` (Eingabe-Halt mit Löschen), `0002: 00000000020` (Stopp), dazu die Konstante `0005: 00000000042`. Streifendatei `band/band5.txt` mit einer leeren Zeile (Vorspann) und `00010101` (die Ziffer 5: Spuren p, 4, 1). Die Eingabeadresse 05 wird mit dem Adressteil 0000 zur Adresse 0005 verknüpft; der AC erhält den **Inhalt** der Zelle 0005, nicht die Zahl 5.

```
Trommeldatei:
#PROT STUFE=0
0000: 00004600030 # K1 anwählen
0001: 00000000011 # Eingabe-Halt mit Löschen
0002: 00000000020 # Stopp
0005: 00000000042 # Konstante: Inhalt der Zelle 0005
Tastenfolge:
H G R
```

Datei `Tests/leser-ziffer.txt`

```
00010101
```

Datei `band/band5.txt`

```
$ python3 c8205_interp.py --leser1 band/band5.txt Tests/leser-ziffer.txt; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Der AC enthält den Inhalt 42 der Zelle 0005. Ohne `--leser1` endet derselbe Test mit der Fehlermeldung aus Kapitel 7. Steht vor der Ziffer die Zeile `11111111` („Irrung Zeichen“, `band/band5i.txt`), ist das Ergebnis dasselbe: Die Zeile wird übersprungen.

```
$ python3 c8205_interp.py --leser1 band/band5i.txt Tests/leser-ziffer.txt; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Mit dem Zeichen : (Zeile `00001101`) und dem Eingabe-Halt `0001: 00000000010` springt das Programm statt zu addieren nach 0015 (Sprung-Umstellung „ja“). Mit der Anwahl `4500 030` statt `4600 030` und `--leser2` liest derselbe Test von Leser 2:

```
$ python3 c8205_interp.py --leser2 band/band5.txt Tests/leser2-ziffer.txt; echo "Rückkehrcode: $?"
=== Test leser2-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Stanzer: Zeichen 5 und Punkt

Programm: `0000: 00004600030` (K1 anwählen), `0001: 00000250030` (Ausgabe der Ziffer 5, Ausgabe-Adresse `025x`), `0002: 00000730030` (Ausgabe des Punktes, `073x`), `0003: 00000000020`.

```
$ python3 c8205_interp.py --stanzer1 out/band.txt Tests/stanzer-zeichen.txt
=== Test stanzer-zeichen (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
$ cat out/band.txt
00010101
00111011
```

Die Datei `out/band.txt` enthält die Zeilen `00010101` und `00111011`. Der Ordner `out/` muss nicht existieren; er wird angelegt. Ohne `--stanzer1` endet der Test mit der roten Fehlermeldung aus Kapitel 7.

Stanzer-Datei gegen eine Soll-Datei prüfen

Soll der Test die gestanzte Datei selbst bewerten, steht die Dateiprüfung im Teil `Prüfungen:` (Abschnitt 3.4). Die Testdatei bringt Gerät und Prüfung mit; die Soll-Datei `soll.txt` liegt neben der Testdatei und enthält die Zeilen `00010101` und `00111011`:

```
Trommeldatei:
#PROT STUFE=0
--stanzer1 out/band.txt
0000: 00004600030
0001: 00000250030
0002: 00000730030
0003: 00000000020
Tastenfolge:
H G R
Prüfungen:
US=1
--pruefe-stanzer1 soll.txt
```

Datei `Tests/stanzer-pruefe.txt`

```
00010101
00111011
```

Datei `Tests/soll.txt`

```
$ python3 c8205_interp.py Tests/stanzer-pruefe.txt; echo "Rückkehrcode: $?"
=== Test stanzer-pruefe (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  Stanzer 1: soll=Tests/soll.txt ist=out/band.txt OK
Test wahr
Rückkehrcode: 0
```

Abweichender Inhalt, fehlende Soll-Datei oder fehlende Ist-Datei ergibt `Test falsch` (Rückkehrcode 6; Beispiel mit `diff` in Abschnitt 6.5). Für die Schreibwerk-Datei gilt dasselbe mit `--sw-ausgabe` und `--pruefe-sw-ausgabe` (Abschnitt 8.1).

K0: Eingabe aus einer Datei

Programm: `0000: 00004700030` (K0 anwählen), `0001: 00000000011` (Eingabe-Halt mit Löschen), `0002: 00000000020`, dazu die Konstante `0007: 00000000007`. Die Datei `band/k0.txt` enthält eine Zeile `7`.

```
$ python3 c8205_interp.py --k0 datei=band/k0.txt Tests/k0-ziffer.txt; echo "Rückkehrcode: $?"
=== Test k0-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000007 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Der AC enthält den Inhalt der Zelle 0007. Mit `SW=1` vor `G` (Schreibwerk statt Tastatur) ist auch das Zeichen `B` eingebbar; an der Pult-Tastatur nicht (Kapitel 7).

8.4 Diagnose, Speicherauszug und Prüfungen

Verkürzter Gruppen-Transport mit Rechtsverschiebung (Test `s1-R1`): Der AC startet mit 6; der Befehl `4001 542` überträgt ihn in die Spur 40xx, solange der AC nicht negativ wird. Der Leerbefehl 164 (Adresse 0020) löst die Diagnose mit `#SPEI-0020 4000-4037` aus.

```
Trommeldatei:
#DOK-0020 s1-R1: GX-Transport mit Rechtsverschiebung
#SPEI-0020 4000-4037
#PROT STUFE=0
0000: 00003000211
0001: 00004001542
0003: 00000000020
0010: 00000020164
3000: 00000000006
Tastenfolge:
H G R
```

Datei `Tests/s1-R1.txt`

> **Achtung** Der Leerbefehl steht in Zelle 0010, nicht in 0002. Nach einem X-Abbruch (verkürzte Gruppen-/Wiederholungsbefehle mit B = 5 oder 7) holt der Rechner den nächsten Befehl ohne Wartezeit aus dem Sektor, der gerade unter dem Kopf liegt: Die Spur kommt aus dem Befehlszähler (hier Spur 00), der Sektor ist der Sektor der letzten bearbeiteten Zelle plus zwei Wortzeiten (hier 4016: Sektor 16 + 32 = 50, modulo 40 = 10, oktal). Der Befehl muss also nicht in Zelle b+1 stehen. Steht er dort, findet der Interpreter in 0010 den Wert 0, also den Sprung nach 0000, und das Programm läuft endlos. Danach läuft das Programm bei 0003 weiter.

```
$ python3 c8205_interp.py Tests/s1-R1.txt; echo "Rückkehrcode: $?"
=== Test s1-R1 (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0020 s1-R1: GX-Transport mit Rechtsverschiebung
#SPEI-0020 4000-4037
4000: 00000000000
4001: 00000000006
4002: 00000000000
… (4003 bis 4015: jeweils 00000000000) …
4016: 00000000003
4017: 00000000000
… (4020 bis 4037: jeweils 00000000000) …
Ende: Unbedingter Stop (US:=1) | Knoten: 95 | BZ=0004 AC=00000000001 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
```

Speicherzeilen gekürzt (die Adressen laufen oktal).

Es werden zwei Zellen geschrieben. In Zelle 4001 landet der AC (6); danach schiebt die Resultatbehandlung ihn nach rechts (AC = 3). Bei R = 1 gilt z_{32} als Vorzeichen: $z_{32} = 1$, der AC zählt also als „negativ“ (X-Bedingung). Operation und Resultatbehandlung werden noch mit einer weiteren Zelle ausgeführt (4016 := 3, AC danach 1), danach bricht der Befehl ab.

Test mit Prüfungen: Der folgende Test lädt eine Zahl, addiert eine zweite und prüft Register, Flags und zwei Speicherzellen (ein Beispiel mit `Test falsch` steht in Abschnitt 6.5):

```
Trommeldatei:
#DOK-0001 Test Ladebefehl und Addition
#PROT STUFE=0
0000: 00003000211 # AC := <3000>
0001: 00003001210 # AC := AC + <3001>
0002: 00000001164 # Leerbefehl, Diagnose #DOK-0001
0003: 00000000020 # Stopp
3000: 00000000100
3001: 00000000045
Tastenfolge:
H G R
Prüfungen:
BZ=0004 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 00000000100
3001: 00000000045
```

Datei `Tests/pruef-wahr.txt`

```
$ python3 c8205_interp.py Tests/pruef-wahr.txt; echo "Rückkehrcode: $?"
=== Test pruef-wahr (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0004 ist=0004 OK
  AC: soll=00000000145 ist=00000000145 OK
  AR: soll=0000 ist=0000 OK
  FE: soll=0 ist=0 OK
  FUE: soll=0 ist=0 OK
  BS: soll=0 ist=0 OK
  US: soll=1 ist=1 OK
  E: soll=0 ist=0 OK
  A: soll=0 ist=0 OK
  V1: soll=0 ist=0 OK
  V2: soll=0 ist=0 OK
  3000: soll=00000000100 ist=00000000100 OK
  3001: soll=00000000045 ist=00000000045 OK
Test wahr
Rückkehrcode: 0
```

Rückkehrcode 0 (`Test wahr`). Dieselbe Testdatei ohne Prüfungen ergäbe 4.

8.5 Trommel über mehrere Tests und eigene Trommeldatei

`trommel-b` schreibt Zelle 3000 nicht, prüft sie aber auf 100; `trommel-a` schreibt dort 100 und hält. Auf leerer Trommel ist `trommel-b` falsch (Rückkehrcode 6); nach `trommel-a` ist es wahr; beide Tests in einem Aufruf ergeben dasselbe. Eine zweite, leere Trommel liegt unter `trommeln/leer.txt` und wird mit `--trommel` gewählt; mit dem Pfad der Standard-Trommel ergibt `--trommel` dasselbe wie der Aufruf ohne den Schalter.

```
Trommeldatei:
#PROT STUFE=0
0000: 00000000020
3000: 00000000100
Tastenfolge:
H G R
```

Datei Tests/trommel-a.txt

```
Trommeldatei:
#PROT STUFE=0
0000: 00000000020
Tastenfolge:
H G R
Prüfungen:
US=1
3000: 00000000100
```

Datei Tests/trommel-b.txt

```
$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py Tests/trommel-b.txt; echo "Rückkehrcode: $?"
=== Test trommel-b (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  3000: soll=00000000100 ist=00000000000 FEHLER
Test falsch
Rückkehrcode: 6
$ python3 c8205_interp.py Tests/trommel-a.txt; echo "Rückkehrcode: $?"
=== Test trommel-a (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Rückkehrcode: 4
$ python3 c8205_interp.py Tests/trommel-b.txt; echo "Rückkehrcode: $?"
=== Test trommel-b (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  3000: soll=00000000100 ist=00000000100 OK
Test wahr
Rückkehrcode: 0
$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py Tests/trommel-a.txt Tests/trommel-b.txt; echo "Rückkehrcode: $?"
=== Test trommel-a (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
```

```
=== Test trommel-b (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  3000: soll=00000000100 ist=00000000100 OK
Test wahr
Rückkehrcode: 0
$ python3 erzeuge_leere_trommel.py --aus trommeln/leer.txt
$ python3 c8205_interp.py --trommel trommeln/leer.txt Tests/trommel-b.txt; echo "Rückkehrcode: $?"
=== Test trommel-b (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  3000: soll=00000000100 ist=00000000000 FEHLER
Test falsch
Rückkehrcode: 6
$ python3 c8205_interp.py --trommel virtuelle_Geraete/Trommel/trommel.txt Tests/trommel-b.txt; echo "Rückkehrcode:
    $?"
=== Test trommel-b (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  3000: soll=00000000100 ist=00000000100 OK
Test wahr
Rückkehrcode: 0
```

Die letzte Zeile zeigt: `--trommel` mit dem Pfad der Standard-Trommel ergibt dasselbe wie der Aufruf ohne den Schalter. Der Wrapper übergibt `--trommel` nicht; eine andere Trommel steht hinter der Testdatei: `./c8205_interpreter.sh Tests/x.txt --trommel andere/trommel.txt` (Abschnitt 4.3).

9 Kurzreferenz

```
python3 c8205_interp.py [Schalter] TEST.txt [TEST2.txt ...]
./c8205_interpreter.sh TEST.txt [Schalter]       # Protokoll nach Test-Protokolle/TEST.log
```

Schalter	Wirkung (Standard)
`--trommel P`	Trommeldatei, bleibt über Tests und Aufrufe erhalten (`virtuelle_Geraete/Trommel/trommel.txt`); leer erzeugen: `python3 erzeuge_leere_trommel.py [--aus PFAD]`
`--mdverz V` · `--out V|F.log`	Regelwerk-Ordner (`markdown-dateien/`) · Protokoll in Ordner `V/<Test>/protokoll.txt` oder Datei `F.log` (nur 1 Test); `Test-Protokolle/`
`--leser1 P --leser2 P --stanzer1 P`	Streifen (Datei/Ordner) für K1/K2 · Stanzerdatei K1; Zeilen `0/1` für w v u p 8 4 2 1
`--sw-ausgabe P` · `--k0 datei=P`	Schreibwerk (K2) in `P/sw-ausgabe.txt` statt ans Terminal · K0-Eingabe aus Datei statt vom Terminal. Alle fünf Geräteschalter auch als Zeile der Testdatei
`--pruefe-stanzer1 D --pruefe-sw-ausgabe D`	Ist- gegen Soll-Datei D vergleichen (setzt das Gerät voraus; nur 1 Test); auch als Zeile in `Prüfungen:`
`--knotentest` · `--NO-TERMOUT`	Knotenzeilen `Kxxx` ins Protokoll (aus) · Ausgaben des Interpreters am Terminal unterdrücken, Groß-/Kleinschreibung beachten (aus)
`--SW-TERMINAL` · `--zwischenablage M`	Ausgabefenster „SW Ausgabe“ (aus; nur Kommandozeile) · Kopierverfahren `auto`, `tk`, `system` (`auto`)
`--version` · `-h`	Version `y.xx-zzz` ausgeben · Hilfetext

Hinweis Fenster „SW Ausgabe“ (`--SW-TERMINAL`)**.** Das Fenster bleibt **immer offen**, auch nach dem Ende des Tests, bis Sie es mit Klick auf X schließen. Es läuft als eigener Prozess; jeder Aufruf öffnet ein neues Fenster. Text im Fenster lässt sich mit der Maus markieren und kopieren: sofort beim Loslassen der Maustaste, mit Strg+C oder Strg+Einfg oder über das Kontextmenü (rechte Maustaste). Das gilt unter Linux und unter Windows.

Marke, Token, Kanal	Wirkung
`#DOK-xxxx T` `#SPEI-xxxx s-e` `#SPUR-xxxx a`	bei Befehl 164 mit Adresse `xxxx`: Text · Speicher `s…e` · ganze Spur von `a`
`#PROT STUFE=n` · `#AC=` · `#BZ=`	0 kein, 1 Bildschirm (Standard), 2 Datei · Anfangswerte AC (11 Stellen), BZ (4)
`#STARTE n` · `#INTERRUPT H …`	`H G M+ E … E M- R` (n-mal `E`), ersetzt die Tastenfolge · Unterbrechung nach der Zeile davor, `R/ST` setzt fort
`H R G`	Halt · Rechnen (H := 0, US := 0, Start) · Generallöschen, danach US := 1, K0; `SW=1` vorher setzen
`M+ M- M E`	M halten/loslassen/antippen (OR v 025); `R` nach `M` mit Sprung mit Rückkehrabsicht: Knoten 54 · `E` Einzelschritt, unter `M`: BZ + 1
`Ü ST SW=1/0 BS=1/0 SP=1/0`	AR := 7777, OR v 240 (nur bei H) · OR v 240, US := 0 (nur bei US/BS) · Schreibwerk/Tastatur · einrastende Schalter
`K0` … `K3` · Anwahl `4700` … `4400` + `030`	Kanaltaste (nach `G`), im Programm übersteuert. K0 Tastatur (T)/Schreibwerk (SW), keine Ausgabe · K1 Leser 1/Stanzer 1 · K2 Leser 2/Schreibwerk (dunkelgelb) · K3 frei (Stub)
`Trommeldatei:`	Zeilen `aaaa: wwwwwwwwww [# …]`, Testmarken, Geräte-Zeilen; nicht genannte Zellen behalten ihren Inhalt
`Tastenfolge:` · `Prüfungen:`	Token, Ende mit `R` oder im Halt · optional: `BZ=… AC=… FE=…`, Zeilen `aaaa: w…`, `--pruefe-…`
Endzeile · Rückkehrcode	`Ende: Grund | Knoten: n | BZ AC AR | FE FUE BS US E A V1 V2` · 0 wahr, 4 ohne Prüfung, 6 falsch, 8 Laufzeitfehler, 16 Vorab-Fehler, 2 ungültiger Aufruf

Anhang A: Glossar

Begriff	Erklärung
Akkumulator (AC)	Rechenregister (33 Bit); einziger Weg zum Abspeichern in die Trommel
Befehlszähler (BZ)	Register (12 Bit) mit der Adresse des nächsten Befehls
Adressregister (AR)	Register (12 Bit), wählt die Trommelzelle für jeden Zugriff
Operationsregister (OR)	Register (9 Bit) mit dem Operationsteil des aktuellen Befehls (BAZ)
Befehlsregister (BR)	Adreßteil (12 Bit) und OR (9 Bit); im Protokoll `BR=aaaa ooo`
Flag	Speicher für 0 oder 1 (E, A, T, SW, K1–K3, FE, FÜ, BS, US, V1, V2, H, SP); eine Lampe zeigt ihn, ein Schalter kann ihn setzen
Trommel	Magnettrommelspeicher: 4096 Zellen à 33 Bit. Im Interpreter eine persistente Textdatei (`trommel.txt`, 4096 Zeilen); ihr Inhalt bleibt über Tests und Aufrufe erhalten
Trommeldatei	die Datei `trommel.txt` (bzw. der Pfad von `--trommel`): der Speicher des Interpreters. Nicht zu verwechseln mit dem Teil `Trommeldatei:` einer Testdatei
Trommelabschnitt	der Teil `Trommeldatei:` einer Testdatei: die Zellen, die der Test vor dem Start auf die Trommel schreibt
Sektor / Bahn (Spur)	32 Sektoren je Bahn; 128 Bahnen. Eine Adresse besteht aus Bahn (7 Bit) und Sektor (5 Bit)
Oktal	Zahlen zur Basis 8 (Ziffern 0–7); alle Zahlen in Testdateien
Knoten, Konnektor	Elemente des Knotengraphen (Anweisung, Test, Phasenzuordnung, Konnektor); der Interpreter läuft ihn ab
Phase I bis IV	die vier Abschnitte eines Befehlszyklus: Zähleraufstellung, Befehlsentnahme, Adressaufstellung, Ausführung
ANWEISUNG-nnn	Beschreibung eines Knotens in Tabelle T2; im Interpreter eine eigene Funktion
Stub	Meldung „an dieser Stelle fehlt eine Umsetzung“ bzw. „außerhalb des definierten Betriebs“; der Test hält an
Testmarke	Zeile mit `#`, die nur der Testdurchführung dient (`#DOK`, `#SPEI`, `#SPUR`, `#PROT`, `#AC`, `#BZ`, `#STARTE`, `#INTERRUPT`)
Makro `#STARTE`	Kurzschreibweise für die Tastenfolge zum Ansprung eines Speicherplatzes (`H G M+ E … E M- R`)
Unterbrechung (`#INTERRUPT`)	Eingriff des Bedieners mitten im Programm: weitere Tasten nach dem Befehl der Zeile davor
Ausgabefenster „SW Ausgabe“	Textfenster, das `--SW-TERMINAL` öffnet; zeigt Schreibwerk-Ausgabe und K0-Echo
Zeichenlage KB/GB	Klein-/Großbuchstaben-Lage; beim Schreibwerk zufällig gesetzt, umgeschaltet durch Umschaltzeichen
Kanal K0 bis K3	Anwahl der Ein-/Ausgabegeräte (Tabelle in Kapitel 5)
T / SW	Subflags für K0: Pult-Tastatur bzw. Schreibwerk statt Tastatur (alternativ)
Sprung-Umstellung	Eigenschaft eines Zeichens der Kodetabelle: Der Eingabebefehl wird in einen Sprung umgestellt (z. B. `:`, `.`)
Irrung Zeichen	Lochbild 11111111 (alle acht Spuren); am Leser von der Paritätsprüfung ausgenommen und übersprungen
Tastenfolge	die Bedienhandlungen am Pult, die den Ausgangszustand festlegen
Dateiprüfung	Vergleich der vom Lauf erzeugten Stanzer- bzw. Schreibwerk-Datei mit einer Soll-Datei (`--pruefe-…`)
Rückkehrcode	Zahl, die der Interpreter dem Betriebssystem liefert (0, 2, 4, 6, 8, 16; Abschnitt 4.4)
Wortzeit	Zeit, in der ein Wort unter dem Kopf durchläuft (104 µs); im Interpreter 0

Anhang B: Grenzen des Interpreters

- Zeiten sind 0: Es gibt keine Wartezeiten und keine Zeitmessung; die Pufferung der Ausgabe (1-Zeichen-Puffer) wird nicht nachgebildet.
- Kanal K3 wird nicht benutzt (Festlegung des Autors); Programme auf K3 und ein Ausgabe-Halt ohne Ausgabegerät liegen außerhalb des definierten Betriebs und enden mit einem Stub.
- Die Schreibwerk-Ausgabe geht ohne `--sw-ausgabe` an das Terminal, mit dem Schalter in die Datei `sw-ausgabe.txt`; mit `--SW-TERMINAL` zusätzlich in ein Fenster. Ein Warten auf den Schalter SW findet nicht statt. Die Stanzer-Ausgabe entspricht dem Regelwerk (eine 0/1-Zeile je Zeichen); den Dateinamen bestimmen Sie mit `--stanzer1`. Der Stanzer hat keine Terminal-Ausgabe.
- Das Ausgabefenster braucht `tkinter` und eine grafische Oberfläche; es läuft als eigener Prozess und schließt sich nie von selbst. Jeder Aufruf mit `--SW-TERMINAL` öffnet ein neues Fenster.
- Die früheren Schalter `--k0 tastatur` und `--leser-ende` gibt es nicht mehr: Ohne `--k0` wird vom Terminal gelesen, Leser und `--k0 datei=` warten am Ende still auf eine neue Datei.
- Ein Transportloch außerhalb des Vorspanns (nach dem Original der Paritätsprüfung unterworfen) ist im Streifenformat nicht darstellbar und nicht umgesetzt; die Auslegung steht noch aus.
- Der Rückkehrcode bezieht sich auf genau einen Test, bei mehreren Testdateien auf den zuletzt bearbeiteten.
- Die Trommel wird nie zurückgesetzt; Tests sind nur dann voneinander unabhängig, wenn sie ihre Zellen selbst nennen oder die Trommel vorher neu erzeugt wird (Abschnitt 3.7). Es gibt keine Sperre gegen gleichzeitige Läufe auf derselben Trommeldatei.
- Im Protokoll (Abschnitt 1) trägt die Kopfzeile noch den Text „nicht genannte Zellen = 0“ aus der Zeit vor der persistenten Trommel; das Protokoll zeigt nur die Zellen des Trommelabschnitts (Abschnitt 6.4).
- Nach einem X-Abbruch (B = 5 oder 7) steht der Folgebefehl nicht in Zelle b+1, sondern im Sektor, der zeitlich fällig ist (Abschnitt 8.4); Tests mit Folgebefehl in b+1 laufen sonst in eine Endlosschleife.
- `B` und `D` (Pult-Tasten) haben keinen Code. `WM` = Tabulator (Eingabe-Adresse 075), `IB` = Eingabe-Adresse 036; die Tabellenzeile 52 („Transport“) wird nicht benutzt.
- Der Interpreter ist wortlautgetreu nach dem Knotengraphen: ANWEISUNG-001 gilt in Phase I und III; die Resultatbehandlung gilt für jeden Befehl außer den Leer-Befehlen 164/166; bei B = 5 und 7 entscheidet bei R ≠ 1 das Bit z_0, bei R = 1 das Bit z_{32} über den X-Abbruch.
- Die Kopfzeile `Prüfungen: False` (Negativtests, Quelle „Durchführung eines Tests“, Abschnitt 3a.5) wertet diese Fassung nicht aus (Abschnitt 3.4).

Anhang C: Verweise auf das Regelwerk

Thema	Dokument
Rechner, Phasen, Register, Trommel	PDF „Rechnerbeschreibung“ (Quellen: Überblick_und_Gesamtaufbau, Datenstruktur…, Magnettrommel)
Tasten, Schalter, Kanäle, Leser, Stanzer, Schreibwerk	PDF „Bedienpult und Peripherie“ (Quellen: Bedienpult_mit_Schaltern, Lochstreifenleser)
Befehle und Formeln	PDF „Einzelbefehlsliste“ (Quelle: Befehlsliste.md)
Zeichenkodes, Tabelle 1	PDF „Kodetabellen“
Knotengraph und Variablentabelle T2	PDF „Knotengraph“ und „Knotentabellen“; Quellen Eingänge_der_Knoten.md, Beschreibung_der_Knoten.md
Tests schreiben, Beispiele, Protokolle lesen	PDF „Testhandbuch“
Testdefinition, Testdatei und Protokoll im Wortlaut	PDF „Durchführung eines Tests“ (Quelle: Durchführung_eines_Tests.md)
Assembler `c82ass` (erzeugt die `.test`-Dateien)	PDF „Assemblerdokumentation“
Lochstreifen erzeugen, anzeigen und umwandeln (für `--leser1`, `--leser2`)	PDF „Lochstreifen-Hilfsprogramme“
Orientierung für neue Leser, Lesereihenfolge der Dokumente	PDF „Einstieg für Neueinsteiger“
Gesamtüberblick des Projekts	PDF „Projektüberblick“
Cache der BAZ-Wirkungen	`baz-wirkungstabelle.md` (vom Python-Interpreter nicht gelesen)